



华南理工大学
South China University of Technology

硕士学位论文

基于 GPU 的实时大气散射渲染优化算法
研究与实现

作 者 姓 名	方 辰
学 科 专 业	软 件 工 程
指 导 教 师	陈 泽 琳
所 在 学 院	软 件 学 院
论 文 提 交 日 期	2017 年 4 月

Real-time Atmospheric Scattering Rendering Algorithm Based on GPU

A Dissertation Submitted for the Degree of Master

Candidate: Fang Chen

Supervisor: Prof. Chen Zelin

South China University of Technology

Guangzhou, China

分类号: TP391

学校代号: 10561

学 号: 201420130852

华南理工大学硕士学位论文

基于 GPU 的实时大气散射渲染优化算法 研究与实现

作者姓名: 方辰

指导教师姓名、职称: 陈泽琳 副教授

申请学位级别: 工学硕士

学科专业名称: 软件工程

论文形式: 产品研发 工程设计 ☒ 应用研究 工程/项目管理 调研报告

研究方向: 计算机图形学

论文提交日期: 2017 年 4 月 20 日 论文答辩日期: 2017 年 6 月 2 日

学位授予单位: 华南理工大学 学位授予日期: 年 月 日

答辩委员会成员:

主席: 李东

委员: 李引、陈泽琳、方琼、徐杨

摘 要

大气散射是自然界中一个非常重要而又普遍存在的物理现象，它使得晴朗的天空在白天呈现蓝色，而在傍晚则显得昏黄；它使得近处的场景显得清晰，而远处的场景则显得模糊。同时，大气散射也是体积光等光学现象的产生原理，对影响人们的视觉感官具有十分重要的作用。

在计算机图形学中，传统绘制天空的方法主要是天空盒、天空贴图等，这种方法简单快速，但却无法模拟出随时间和气候实时变化的动态光照效果。随着计算机硬件性能特别是 GPU 图形渲染能力的提升，在虚拟场景中模拟出基于真实物理的大气散射效果逐渐成为了可能。对此，人们已经总结出了一个完整的散射积分方程，可以十分准确的计算出光线在空气中传输的辐射度变化。然而，由于该积分的计算十分复杂，而且需要对场景空间中的每一个点都进行积分计算，运算量非常庞大，即使是在现代的 GPU 上依然很难做到流畅的实时渲染。

在这样一种背景下，本文实现了一个基于 GPU 计算的实时大气散射渲染优化算法。通过对大气进行基于真实物理的建模，利用光线步进的思想计算散射积分，并使用预计算查找表来简化散射积分计算、通过极线采样来降低样本点的数量、利用一维最小/最大二叉树结构来加速可见因子的判断。算法在保证了真实的画面渲染效果的同时，显著的提升了渲染效率。

本文首先阐述了算法所建立的空气物理模型，并详细的说明了散射积分的推导过程，然后，我们描述了算法是如何一步步进行优化的，并展示了在 GPU 上的实现细节。最后，我们编写了一个测试程序用于展示渲染效果，并与传统的算法做出了对比，证明我们的算法无论在渲染效果和渲染效率上，都能够得到很好的结果。

关键词：计算机图形学；大气散射；体积光；光线步进；极线采样；

ABSTRACT

Atmospheric scattering is a very important and common physical phenomenon in nature, it makes the sky seems blue in the daytime, and seems yellow in the evening; It makes the near scene seems clear, and the distant scene seems dim. Atmospheric scattering is also the principle of a lot of optical phenomenon, such as volume light. It plays an important role in the influence of people's visual sense.

In computer graphics, the traditional drawing method of the sky is sky box or sky map. This method is simple and fast, but it cannot simulate the dynamic lighting effects of real-time climate change in different time. As the improvement of computer performance especially the GPU graphics rendering ability, atmospheric scattering effect based on real physical simulating in the virtual scene gradually become possible. Therefore, people have summarized a complete scattering integral equation, which can calculate the light radiation intensity changes in air transport accurately. However, due to the complication of the integral calculation, even in the modern GPU is hard to hold smooth real-time rendering.

Under such a background, this thesis implements a real-time atmospheric scattering rendering optimization algorithm based on GPU computation. We calculate the scattering integral by ray-marching based on the real physical modeling of the atmosphere, and use a pre-computed lookup table to simplify the scattering integral calculation, use epipolar sampling to reduce the number of sample points, using a 1D min/max binary tree structure to accelerate the visible factor judgment. As ensuring a good rendering result, we significantly improve the rendering efficiency.

This thesis first expounds the algorithm of the air physical model, and detailed description of the derivation process of the scattering integral, then, we describe the algorithm how to optimize the calculation step by step, and shows the implementation details on the GPU. Finally, we write a test program for rendering effect, and made a comparison with the traditional algorithm, our algorithm can get a satisfying result, both on the rendering result and rendering efficiency.

Keywords: Computer Graphics; Atmospheric Scattering; Volume Light; Ray Marching; Epipolar Sampling;

目 录

摘 要.....	I
ABSTRACT	II
目 录	III
第一章 绪论	1
1.1 选题背景及研究意义.....	1
1.2 研究现状	2
1.3 主要研究内容	5
1.4 论文章节安排	5
第二章 论文相关理论基础	7
2.1 大气散射的基本概念.....	7
2.2 空气的物理模型	7
2.2.1 瑞利散射	8
2.2.2 米氏散射	9
2.2.3 大气密度	11
2.3 光线散射积分	11
2.3.1 光学深度	13
2.3.2 散射系数	13
2.3.3 相位函数	13
2.3.4 可见因子	14
2.4 多重散射	14
2.5 体积光	15
2.6 本章小结	16
第三章 光线散射积分优化算法设计	17
3.1 光线步进和梯形法则.....	17
3.2 预计算查找表	18
3.3 极线采样	20
3.3.1 极线生成.....	21

3.3.2 样本生成	22
3.3.3 样本细化	23
3.3.4 可见因子与阴影贴图	23
3.3.5 光线步进	25
3.4 一维最小/最大二叉树	25
3.4.1 极切片	25
3.4.2 创建最小/最大二叉树	27
3.4.3 使用一维最小/最大二叉树检测可见因子	28
3.4.4 像素插值	30
3.5 本章小结	31
第四章 散射积分优化算法的 GPU 实现	32
4.1 DIRECTX 11 介绍	32
4.1.1 DirectX 简介	32
4.1.2 DirectX 的组件构成	32
4.1.3 Direct3D	32
4.1.4 DirectX 11 的新特性	33
4.2 算法整体流程	34
4.3 极线采样	36
4.3.1 极线和初始样本生成	36
4.3.2 间断点检测和样本细化	37
4.4 构建一维最小/最大二叉树	38
4.5 光线步进	39
4.6 像素插值	41
4.7 本章小结	41
第五章 实验结果与讨论	42
5.1 配置表选项	42
5.2 渲染效果展示	44
5.3 渲染效率测试	50

5.4 本章小结	53
总结与展望	54
参考文献	55
攻读硕士学位期间取得的科研成果	58
致谢	59

第一章 绪论

1.1 选题背景及研究意义

大气散射是自然界中一个非常重要而又普遍存在的物理现象，它使得晴朗的天空在白天呈现蓝色，而在傍晚则变得昏黄；它使得近处的场景显得清晰，而远处的场景时显得模糊（图 1-1）。体积光、云隙光等光学现象同样产生于该原理（图 1-2）。如果没有大气散射，那么就只有被太阳光直接照到的物体是明亮的，而被阴影遮挡的物体则是完全黑暗不可见的，这显然并不符合人们的认知。所以，通过计算机在虚拟场景中模拟出大气散射效果对增强场景的真实性具有非常重要的意义。

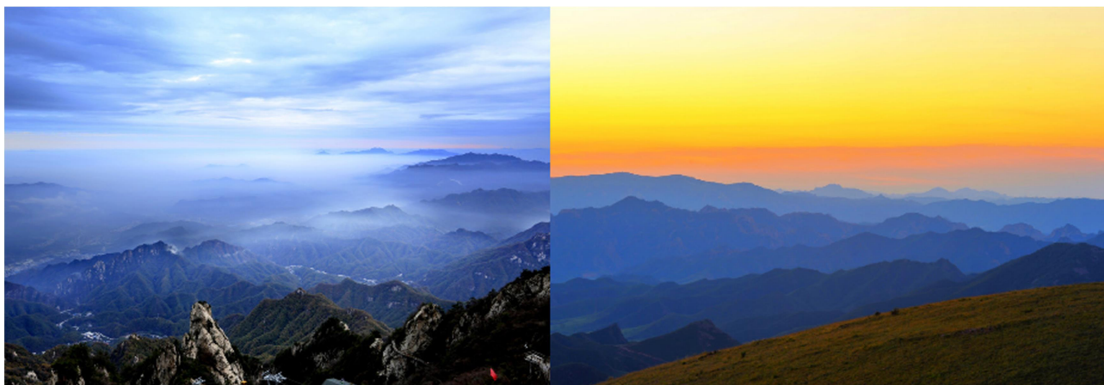


图 1-1 自然界中的大气散射现象

随着计算机硬件性能特别是 GPU 图形处理能力的飞速发展，计算机已经可以越来越逼真的模拟出真实的物理世界，科学家和工程师们也在不断的提出各种图形学渲染算法，以满足用户对场景真实性的追求。而在对天空及室外光的模拟方面，早期的虚拟现实及三维游戏应用由于硬件机能的限制，只能使用一个简单的天空盒模型来进行模拟。这种方法的优点是简单，整个天空仅仅通过五张纹理贴图来实现，并且可以通过许多技术来模仿出动态效果。但天空盒模型在本质上仍然是静态的，并且是完全物理无关的，所以具有很大的局限性。随着基于物理的光照模型在计算机中越来越深入的应用，人们需要一种更加真实的、完全动态的、并可以从物理本质上模拟出自然界天空光照效果的渲染模型。为此，人们提出了一种基于物理规律的大气散射光照模型。

大气散射产生的主要原因是太阳光与空气中的分子及气溶胶的相互作用，光线在大气层内受大气分子的影响，经过多次的衰减和散射，最终到达人的眼睛。所以，最终人们观察到的光照效果，与太阳光本身所发出的光线特点已经大不相同。例如蓝色光由于

波长较短，其散射作用更为明显，所以人们在晴朗的白天所观察到的天空是蓝色的；而在傍晚，由于蓝色光经过太多次散射已经无法到达人的眼睛，所以人们观察到的天空颜色是波长更长的橙红色。

为了精确的计算大气散射，物理学家们已经研究并总结了一套完整的积分公式，该公式综合考虑到了光照的强度和方向、光线的散射和衰减、光源与视点的距离、以及大气中粒子浓度的影响。而在计算机模拟中，依托于 GPU 的并行计算特点，我们可以同时对屏幕的每个像素点都对这个十分复杂的积分公式进行一次计算。但由于该积分计算的复杂性，以及实时渲染对性能的高要求，即使是在 GPU 计算能力已经十分强大的现在，想要直接计算出高精度的大气散射结果依然是很困难的。所以，我们必须要通过算法优化这一计算过程，使其在有限的硬件机能条件下，可以做到在实时渲染出逼真画面的同时又能够保持较高的渲染性能，这是一个十分值得研究并具有挑战性的问题。



图 1-2 计算机模拟的体积光现象

在这样一种背景下，本文在总结前人研究工作的基础上，设计并实现了一种基于 GPU 计算的实时大气散射渲染优化算法，该算法在实现了基于物理的空气光散射积分计算的基础上，通过预计算查找表、极线采样、与一维最小/最大二叉树优化了散射积分的计算，在保证真实的渲染画面前提下，同时提供了很好的渲染性能。

1.2 研究现状

在上世纪 70 年代初，由于光栅扫描显示器的出现，使得计算机图形学进入了一个飞速发展的阶段，许多经典的图形学算法相继被提出，而其中十分重要的就是光照模型理

论的出现。由 Bouknight 等人在 1970 年提出的第一个反射光模型——“Lambert 漫反射模型”^[1], 开启了真实感光照的研究热潮。紧接着在 1971 年, Gourand 又提出了“Gourand 明暗插值模型”的漫反射+插值的算法思想^[2]。1973 年, Phong 提出了著名的简单光照模型“Phong 模型”, 直至今日依然被应用于一些简单光照的场合^[3]。而到了 80 年代, 人们开始研究室外场景的光照渲染, 于是各种关于天空渲染的光照模型相继被提出。

Nishita 等人在 1986 年提出了一个基于经验的天空模型, 在这一模型中天空被划分为若干个等距的条状光源, 可以模拟出晴朗和阴雨天气下的太阳光照效果, 具有一定真实性但却是完全物理无关的, 所以在很多地方并不准确^[4]。次年, Nishita 等人又提出一种单重散射模型, 该模型考虑到了不同光照强度的分布, 采用了均匀大气密度分布的假设, 并使用光照体和阴影体来渲染光线射入介质时的效果^[5]。Dobashi 等人在 1994 年对[4]的方法做出的改进, 提出了使用预计算不同纬度的太阳光强, 按一定间隔采样并存储为函数权值的形式, 可以较快速的渲染出天空的颜色^[6]。但由于这些方法是都基于经验模型的, 所以具有很大的局限性。

Klassen 在 1987 年提出了一种大气分层模型, 该模型将大气层分为许多互相平行的平面, 并假设每层的空气密度相同, 而且还考虑到了空气分子和空气溶胶对光线散射的不同影响^[7]。尽管这样的模型与实际的大气分布差异较大, 但却是第一个基于空气物理的大气光照模型, 所以得到了后人的许多研究和改进。Inakaged 在 1989 年改进了 Klassen 的方法, 使得模型可以使用近似几何光学来处理大粒子^[8]。Kaneda 在 1991 年同样对[7]中的模型提出了不同的改进, 他们不再假设大气是分层的平面, 而是将其看作是一种类似球状的外壳, 并认为空气粒子密度是按离地面的高度呈指数分布的^[9]。Nishita 等人在 1993 年对 Kaneda 的方法做出了进一步的改进, 考虑到了瑞利散射的影响, 并给出了具体计算公式^[10]。Irwin 在 1996 年同样提出了一种基于空气分子瑞利散射的方法来渲染大气效果^[11]。然而, 这两种方法都具有同样的局限性, 即对空气中大颗粒所造成的米氏散射仅仅只是使用经验函数去逼近, 误差较大, 因此只能用于表现晴朗纯净的天空环境。Jackel 等人在 1997 年对大气的成分做出了具体的划分, 如纯净空气、臭氧层、尘埃、雨滴等, 充分考虑到了大颗粒所产生的米氏散射对光线的影响, 因此可以模拟不同气候环境下的大气散射现象^[12]。以上的方法都是基于真实的物理模型, 因此可以绘制出较为准确的天空场景, 但由于都使用了密集采样和复杂的积分计算, 所以渲染效率很低。

早期的实时渲染算法因为计算机硬件机能的限制，只能做非常简单的模拟。在基于平坦地面、恒定大气密度和光照强度的前提下，Hoffman 和 Preetham 在 2002 年推导出了一个完全解析的散射光积分模型^[13]。该模型可以得到正确的地表显示效果，但对天空的模拟仍会显得不自然，特别是在近地平面处。Riley 在 2004 年提出了一种更精确的模型，在他的假设中地面是平坦的，但大气密度却是以指数形式递减的^[14]。然而为了获取一个闭合的散射光积分，他忽略了光线在空气溶胶上的散射（米氏散射）。Preetham 等人在 1999 年曾试图使用解析函数来逼近复杂的大气效果^[15]。Mitchell 在 2008 年则尝试使用屏幕空间射线模糊算法来模拟体积光^[16]。

为了加速积分计算，人们提出了各种不同的方法，主要都是针对积分公式中的光学深度的计算。1993 年，Nishita 等人基于太阳光是平行光这一假设，使用了一张预计算的二维查找表将散射积分的计算量降低了一半^[10]。Dobashi 等人在 2002 年使用了类似的预计算方法进一步降低了光学深度的计算量^[17]。而 O'Neil 在 2004 年发现了一种更快速的预计算方法，他所使用的查找表可以使得程序不需要对外散射的两个光学深度积分再做实时计算^[18]。O'Neil 在后来又使用了一个解析函数来替代查找表^[19]。而 Schuler 在 2012 年基于对查普曼函数的近似提出了一个相当准确的解析方程^[20]。

还有一些研究试图对散射积分进行尽可能多的预计算来加速实时渲染过程。一般来说散射积分公式取决于四个变量，Schafnitzel 等人拿掉了其中一个参数并将积分存储在一个三维纹理中^[21]。Bruneton and Neyret 在这个方法的基础上，使用了更复杂的四维参数表。他们的算法同时模拟了多重散射和地表反射，通过一个三维纹理数组存储结果数据，然后再将其插值到四维坐标系中^[22]。为了渲染体积光，他们使用了阴影体来计算视线被照明部分的全长，然后计算最终的内散射积分，并假设视线被照明部分是连续的并且起始点是摄像机位置。

Engelhardt 和 Dachsbacher 在 2010 年发明了一种优雅而且高效的算法，用于渲染均匀散射介质中的散射效果^[23]，他们注意到了内散射光强度是随着自光源投影点到屏幕边缘的极线均匀变化的。为了解释这一点，他们提出了一种算法，将光线步进样本沿着极线进行稀疏采样，并在适当的地方进行插值，最后，通过在深度不连续的地方放置额外的样本以保持插值的正确性。

Chen 等人在 2011 年同样利用了极几何的特点，他们进一步意识到可以对每个极切

片建立一维最小/最大二叉树来加速光线步进的过程。他们的算法更加复杂和精确，基于对散射项做奇异值分解，并且需要对近核点的区域做特殊的校正^[24]。Yusov 在 2013 年也曾利用了和 Chen 同样的思想，但在实现细节上有所不同^[25]。

1.3 主要研究内容

本文的主要研究目标是，在基于大气物理模型的前提下，设计并实现一个高效的实时大气散射光照渲染算法。其主要研究内容如下：

（1）研究空气的物理特性，分析空气中不同粒子成份的组成特点，以及空气密度在不同海拔高度上的变化规律，在计算机中对大气进行基于物理的建模。

（2）研究太阳光在大气层中传播时与不同大气微粒交互的表现，结合前人的工作，对散射光积分公式进行详细的推导，并分析其中重要参数的作用。

（3）利用积分的梯形法则，通过光线步进的思想，在 GPU 上实现对散射光的积分计算。

（4）对于积分中十分耗时的光学深度计算过程进行简化，具体方法是通过建立一张预计算的二维查找表，将复杂的积分计算过程转化为更快速的直接查找过程^[18]。

（5）通过极线采样的思想，将原本每像素的密集计算过程转化为采样计算与插值的过程，大幅度的减少了算法的总体计算量^[23]。

（6）利用一维最小/最大二叉树的数据结构，将原本的通过遍历阴影贴图纹理判断可见因子的过程，转化为更为高效的二叉树判别过程，进一步加快了光线步进的计算速度^[24]。

（7）最后，使用 DirectX11 图形接口设计并实现一个测试程序，用于展示算法的渲染效果、测试渲染效率，并与前人的算法做对比。最终证明了本文所提出的算法无论在效果和效率上，都能够得到很好的结果。

1.4 论文章节安排

第一章为绪论，主要从整体上介绍了论文的主要研究内容和整体结构。

第二章主要介绍论文所使用到的相关理论基础。

第三章主要描述基于预计算查找表、极线采样和一维最小/最大二叉树的散射积分计算优化算法。

第四章主要描述本算法在 GPU 上的并行实现细节。

第五章主要介绍实验结果分析，测试渲染画面，对比渲染性能。

第六章主要对本次研究做出了总结，并提出了对未来相关工作的展望。

第二章 论文相关理论基础

2.1 大气散射的基本概念

大气散射是指，太阳光在射入大气层时，与大气中的空气分子或空气溶胶等发生相互作用，使得入射的光能以一定的规律在各个方向上进行重新分布的现象。它的本质是由于空气微粒或空气溶胶等粒子被入射光波所影响，发生多极子或电偶极子振荡，使得该粒子以自身为源点向各个方向发散出和入射光波长相同的子光波，称为散射光波。散射光波的特征分布取决于散射粒子的形状、大小、散射率，同时也与入射光线的波长、强度相关。

太阳光在射入大气层时，遇到大气分子、尘埃、雨滴等颗粒后，都会发生散射现象。其中一部分的光能会被这些粒子吸收转化为热能，而另一部分光能则会以该粒子为中心，向四面八方扩散开来。所以，在经过了大气的散射作用之后，有部分太阳光将无法抵达地球表面。

大气散射在自然界中是一种十分重要而又普遍存在的物理现象，人们平时用肉眼观察到的光很大一部分都是散射光。如果不存在大气散射，那么只要不是太阳光直接照射到的位置，都将是完全黑暗的，这显然不符合人们的认知。大气散射使得来自太阳的直射光能被削弱，但又令地面接收到了来自空气散射的间接光能，这就在很大程度上增加了研究该现象的复杂度。大气散射是大气辐射学和大气光学的重要内容之一，也是激光雷达、微波雷达等探测手段的理论基础。由于它的普遍存在性和对人类感官的重要影响，所以在计算机图形学中，如果能够在虚拟场景中很好的模拟出大气散射现象，对提升整个场景的真实感具有非常重要的意义。

2.2 空气的物理模型

太阳光在大气中传播时会被空气介质中的不同颗粒影响，从而使部分光线改变方向，在原传输方向上的光强变弱，这种现象就是光线散射。而空气中的颗粒根据其直径大小的区别可分为两种：直径远小于光线波长的空气微粒，与直径与光线波长相当的空气溶胶。这两种不同粒子的光线散射特性也有很大的差别，由前者引起的散射我们称为瑞利散射，它是导致晴朗天空呈现蓝色的主要原因（如图 2-1 左）。由后者引起的散射我们称为米氏散射，它是导致阴霾的天空呈现灰色的原因，因为阴天的空气中存在大量与光

线波长直径相当的水滴（如图 2-1 右）。



图 2-1 瑞利散射与米氏散射的影响

2.2.1 瑞利散射

由空气中远小于波长的微粒（如空气分子）引起的散射称作瑞利散射(Rayleigh Scattering)。这种现象由英国物理学家瑞利勋爵在 1900 年发现，因此得名。

瑞利散射强度十分取决于粒子的大小（与直径呈六次方正比）。同时，它又与光线波长呈四次方反比，这意味着白光中波长较短的颜色光（蓝色）会比波长较长的光（红色）有更强的散射强度（强 10 倍左右）。所以，瑞利散射是导致天空在白天偏向蓝色，而在黄昏偏向橙红色的根本原因。

在晴朗的白天，太阳处于天空中央，当阳光射入大气层时，就会与空气分子（其直径远小于可见光波长）发生瑞利散射作用。由于蓝色光的波长比红色光短，所以瑞利散射效果更加明显，被散射的蓝光将散布在各处，使得整个天空变为蓝色。而在太阳本身及其周围的空间，由于直射光强度要远大于散射光，所以会呈现太阳光原本的颜色白色。

当日出或日落的时候，由于太阳的位置接近地平线，阳光斜射入大气，会在大气层中穿过很长的距离。在这个过程中，太阳光中的蓝色光几乎都会被散射殆尽无法抵达人眼，只剩下了波长较长的红色光，所以在太阳及其周围的天空都会呈现橘红色，周围的云彩也会因为反射太阳光而呈现红色，这就是朝霞和晚霞出现的原因。而远离太阳的天空，仍会是一种较深的蓝黑色，因为还有少量蓝色光会通过大气散射，被人眼所观察到。

另外，在月球上，因为没有大气层的存在，所以即使是在白天，月球上所观察到的天空仍然是黑色的（如图 2-2）。



图 2-2 月球上的天空

瑞利散射的散射系数可以使用公式 2-1 计算^[11]:

$$\beta_R(\theta) = \frac{2\pi^2(n^2-1)^2}{3N\lambda^4} p_R(\theta) \quad (2-1)$$

其中 θ 是视线与太阳光线的夹角, N 是大气分子的密度, n 是大气的折射率, λ 是入射光的波长, $p_R(\theta)$ 是单位化的相位函数。可以看到, 瑞利散射是明显波长相关的 (与波长的四次方呈反比), 所以在我们的实现中可以将其使用一个 RGB 向量来表示, 其散射系数可表示为^{[14][22]}:

$$\beta_R.rgb = (5.8, 13.5, 33.1) \times 10^{-6} \quad (2-2)$$

同时, 由于瑞利散射几乎是各向同性的, 即光线会被粒子向各个方向均匀散射 (图 2-3 左)。其相位函数可以表示为:

$$p_R(\theta) = \frac{3}{16\pi} (1 + \cos^2 \theta) \quad (2-3)$$

Nishita 和 Preetham 分别在他们的研究中给出过对于该公式的精确推导, 这里不再进行赘述, 有兴趣的可以去查阅文献[10]和文献[15]。

2.2.2 米氏散射

在空气中直径与波长相当的微粒 (如尘埃、雾滴等) 所导致的散射现象称作米氏散射(Mie Scattering)。该理论由德国物理学家古斯塔夫·米在 1908 年提出, 因此得名。

米氏散射与瑞利散射不同, 它几乎是波长无关的, 并且表现出明显的各向异性, 即光线会被粒子更多的向后方散射。在晴朗干净的天空环境下, 由于空气中大颗粒数量较少, 米氏散射的表现并不明显。而当阴雨天气时, 空气中存在大量的水滴颗粒, 米氏散

射就会导致天空呈现灰白色。又如现今经常出现的雾霾天气，同样是因为空气中悬浮的大颗粒过多而导致的米氏散射现象。

另外，云层中充满了大量的直径大于太阳光波长的水滴，太阳光在照射到云层时同样会发生米氏散射，因而云层会呈现灰白色。而海水的颜色同样可以用米氏散射来解释，在远离陆地的远洋中，由于海水的水质较纯净，大颗粒物质较少，米氏散射作用不明显，所以会呈现深蓝色；而在近海区域，由于海水浑浊，含有较多的无机或有机悬浮物，米氏散射现象明显，所以海水会显蓝绿色或黄褐色。

由于米氏散射可以认为是波长无关的，所以在实现中可以使用标量进行表示。米氏散射的散射系数为^[22]：

$$\beta_M \cdot rgb = 2.0 \times 10^{-5} \quad (2-4)$$

与瑞利散射不同，米氏散射的方向是各向异性的，光线会被更多的向后方散射（图 2-3 右），其相位函数为^{[10][14]}：

$$p_M(\theta) = \frac{1}{4\pi} \frac{3(1-g^2)}{2(2+g^2)} \frac{1+\cos^2\theta}{(1+g^2-2g\cos\theta)^{3/2}} \quad (2-5)$$

米氏散射方程的表示比瑞利散射要复杂的多，在实现中可以用 Cornette-Shanks 函数来进行模拟^[18]。在公式 2-5 中， θ 是光线方向与视线方向的夹角，而 g 表示散射的对称性（由粒子的大小决定），若 g 是正值，说明大多数光线会被粒子向后方散射，若 g 是负值，则说明更多的光线会被向前方散射。通常来说， g 的取值一般在 -0.75 到 0.99 之间^[19]，在本文的实现中，我们取 $g=0.76$ 。^[22]

瑞利散射从本质上说也是米氏散射的一个子集，当粒子直径趋于无穷小时，即 g 趋于 0 时，米氏散射方程（公式 2-5）即退化为瑞利散射方程（公式 2-3）。

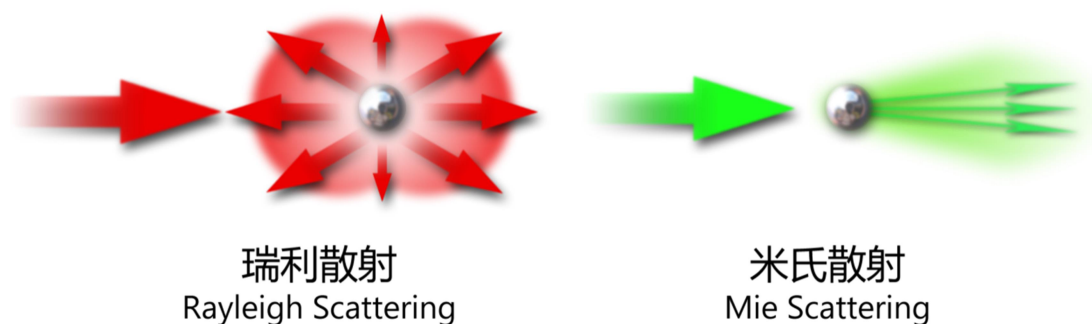


图 2-3 瑞利散射与米氏散射的方向特性

2.2.3 大气密度

对于瑞利散射和米氏散射，它们对太阳光的散射作用都和空气粒子的密度有关。许多大气模型都假设摄像机总是在地面上或者是在十分接近地面的位置，这样就可以认为空气具有一个恒定的粒子密度，这就在很大程度上简化了 Nishita 在 1993 年提出的散射积分方程^[10]，并在近地空间可以得到很好的渲染效果^[15]。然而在远离地表的高空，这种做法得到的渲染结果并不准确。

物理中实际的大气密度并非是个定值，而是在地球引力的作用下，越靠近地表空气密度越高，越远离地表则空气越稀薄。所以，在我们的大气模型中，我们假定空气粒子的密度是沿海拔高度 h 呈指数式递减的：

$$\rho = \rho_0 e^{-h/H} \quad (2-6)$$

ρ_0 是在海平面的空气浓度， h 为当前采样位置的海拔高度， H 是缩放高度（在实现中可设为大气层高度）。理论上说大气层并没有确定的高度，但在实现中我们需要一个统一高度来渲染天空穹顶。在本文的实现中，对于瑞利散射与米氏散射我们分别使用不同的缩放高度： $H_R=7994\text{m}$ ， $H_M=1200\text{m}$ 。^[10]主要原因是影响米氏散射的大颗粒（尘埃、水滴等）更多的存在于近地表的对流层中，再往高空米氏散射效果不明显，但瑞利散射的作用依然存在。由于空气对太阳光的散射和吸收强度都是和空气粒子密度成比例的，所以对于公式 2-6 在 h 点处的散射系数和吸收系数都是在海平面处的近似系数乘以因子 $e^{-h/H}$ 。

2.3 光线散射积分

太阳光在大气中传输的时候会与空气中的微粒产生交互作用。有两种重要的交互方式：散射（图 2-4 左），它改变了光线的方向；吸收（图 2-4 右），它将光能吸收并转变为其它形态的能量（如热能）。而散射效果对场景中物体的影响又分为两个方面，一方面是一部分由物体反射的光被散射到视线之外，并不能到达摄像机，因而被衰减。另一方面是一部分太阳光被空气中的粒子散射正对向摄像机，这些正朝向视线的散射被称作内散射（图 2-4 中）。



图 2-4 空气介质对光线的影响

所以，最后抵达摄像机并被人眼所观察到的光线可以分为两部分：衰减后的物体反射光和被内散射的大气散射光（如公式 2-7 所示）。

$$L_{Cam} = L_{Obj} \cdot e^{-T(O \rightarrow C)} + L_{In} \quad (2-7)$$

其中， L_{Cam} 为最终抵达摄像机的总光强， L_{Obj} 为物体的反射光， $e^{-T(O \rightarrow C)}$ 为反射光从物体位置 O 到摄像机位置 C 的衰减系数，其中 T 被称作光学深度(Optical Depth)或光学厚度(Optical thickness)，它是散射系数在整条路径上的积分。对于密度恒定的单一散射介质来说，我们可以认为它就是路径的长度乘以散射系数^[15]，但在本文的实现中，空气粒子密度是随海拔高度递减的，所以光学深度需表示为散射系数在从 O 点到 C 点路径上的积分和。

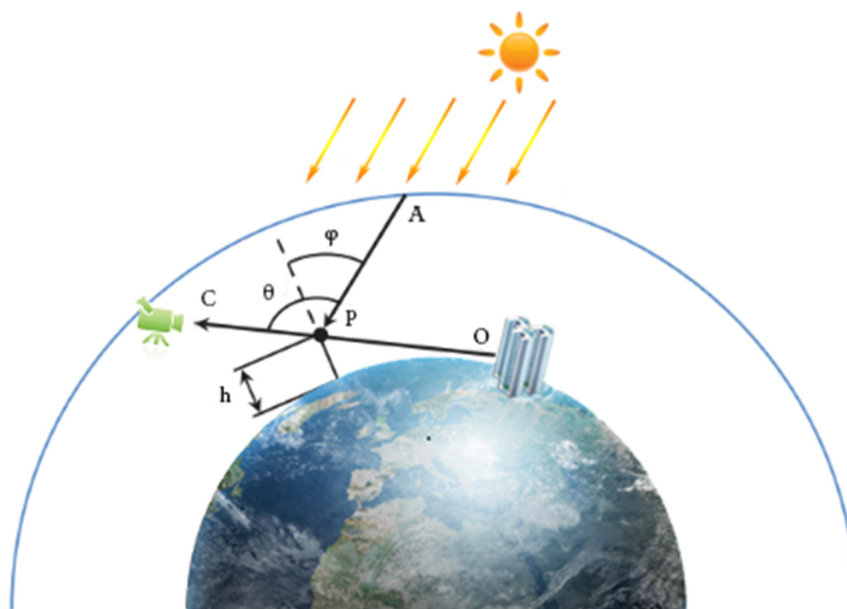


图 2-5 大气散射示意图

2.3.1 光学深度

如图 2-5，假设在大气层内有一点 P，它在从摄像机位置 C 开始到物体位置 O 结束的视线上。阳光从太阳照向地球，在穿过大气层时，会受空气分子和空气溶胶的散射作用影响而发生衰减，最终到达 P 点处的光能总量可以使用公式 2-8 来计算：

$$L_P = L_{Sun} e^{-T(A \rightarrow P)} \quad (2-8)$$

其中 L_{Sun} 是太阳光到达大气层前的初始光能，散射光的强度与 L_{Sun} 的强度呈正比。A 点是光线到达 P 点之前与大气层的交点， $T(A \rightarrow P)$ 被称作从 A 点到 P 点的光学深度，它本质上是 A 点到 P 点整条路径上散射系数的积分，可以用公式 2-9 表示：

$$T(A \rightarrow B) = \int_A^B (\beta_R^e e^{-h(t)/H_R} + \beta_M^e e^{-h(t)/H_M}) dt \quad (2-9)$$

2.3.2 散射系数

在公式 2-9 中，空气分子和空气溶胶的作用是独立计算的， β_R^e 和 β_M^e 分别是瑞利散射和米氏散射的散射系数。散射系数决定了散射介质对光线的散射的强弱程度，也反应了光线在通过该介质时的衰减程度。在 2.2.1 和 2.2.2 的描述中，我们已知瑞利散射对不同波长的光线散射强度不同，在实现中我们可以将其在海平面处的散射系数设为一个三维向量： $\beta_R^e = (5.8, 13.5, 33.1) \times 10^{-6}$ ；而米氏散射对波长的变化影响不明显，所以可以将其在海平面处的散射系数设为一个标量： $\beta_M^e = 2.0 \times 10^{-6}$ 。而又如 2.2.3 节所述，在本文的实现中，空气粒子密度是随海拔的升高呈指数递减的，所以在不同高度处的散射系数也不相同，需再乘以缩放权值 $e^{-h/H}$ 。

2.3.3 相位函数

已知了入射光能和介质的散射系数，我们就可以计算出有多少光线会被介质散射出去。但并非所有的光能都会被散射朝向摄像机，有一部分会被散射到其它方向，无法被肉眼所观察到（我们称为外散射）。所以为了计算内散射的光线量，我们还需要有另外一个因子。

相位函数 $p(\theta)$ 决定了该点有多少光线散射朝向摄像机点，它的参数是太阳光到点 P 的向量 LP 与点 P 到摄像机位置的向量 PC 的夹角 θ （如图 2-6）。另外，相位函数是标准化的，即函数本身在整个方向上的积分为 1。

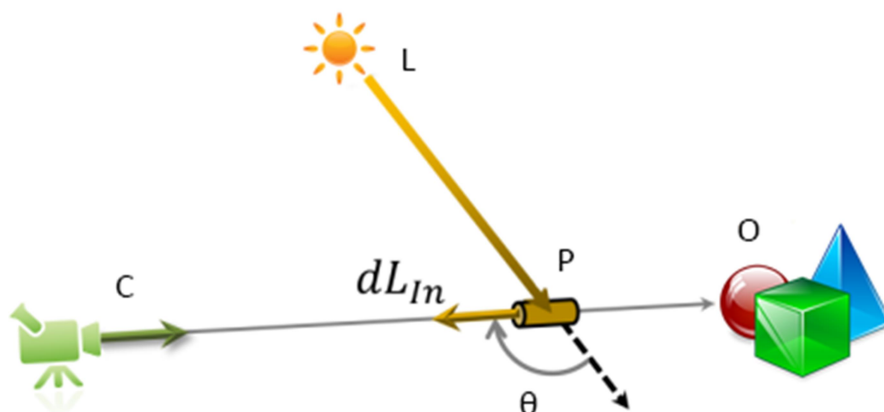


图 2-6 相位函数对散射光的影响

如 2.2 节所述，瑞利散射几乎是各向同性的，光线会以介质粒子为中心均匀的向各个方向散射，其相位函数可以用公式 2-4 来进行表示。而米氏散射则是明显各向异性的，光线会被更多的介质粒子向后方散射，其相位函数可以用公式 2-6 来表示。

2.3.4 可见因子

光线在到达摄像机前可能会被阴影遮挡，为了考虑阴影的影响，我们使用一个可见因子 $V(P)$ 来表示 P 点是否被照亮， $V(P)$ 为 1 时表示该点处于光源的直射下，为 0 则表示该点处于阴影中。在本文的实现中，对可见因子为 0 的点直接过滤，不进行散射积分计算。

最后， P 点处的内散射光在到达摄像机前还会按照因子 $e^{-T(P \rightarrow C)}$ 衰减。则最终的内散射光方程可以用公式 2-10 来表示。

$$L_{In} = \int_C^O L_{Sun} \cdot e^{-T(A(s) \rightarrow P(s))} \cdot e^{-T(P(s) \rightarrow C)} \cdot V(P(s)) \cdot (\beta_R^s e^{-h(s)/H_R} p_R(\theta) + \beta_M^s e^{-h(s)/H_M} p_M(\theta)) ds \quad (2-10)$$

然后我们再将衰减的物体反射光加上，即可渲染出最后摄像机所观察到的最终图像（公式 2-7）。

2.4 多重散射

光线在传输过程中被空气中的一个粒子影响，称为光的一次散射。当空气中大颗粒较多时，被粒子散射的光又会被散射方向上的其它粒子再次散射，这个过程称为多重散射。在晴朗干净的天空中，由于空气中大粒子的数量较少，多重散射的作用不是很明显。而在空气浑浊或黄昏时，多重散射会对场景的真实性产生较明显的影响。

在我们空气光积分推导中，我们将遵从单一散射模型：它假设太阳光在到达摄像机

之前只会进行一次散射。这一假设在白天的时候比较合理，因为在白天的时候太阳光强度较高，多重散射作用不明显；而在傍晚的时候，由于太阳直射光强度变弱，多重散射对场景的影响会变得更加重要，在渲染真实图像中必须加以考虑。即便如此，单一散射模型在此时依旧可以提供一个相对较好的结果。

在我们的了解中，唯一的一个多重散射模型是由 Bruneton 和 Neyret 在 2008 年提出的^[22]。它需要一个非线性参数的四维查找表，而在该表中实时的使用多个查找项是非常耗性能的，所以出于效果和性能的综合考虑，本文并没有采用这样的做法。

2.5 体积光

当光线照射到遮挡物时，一部分光线会从物体的边缘和空隙中穿过，并产生很明显的光柱效果，在视觉上给人以很强的体积感，所以称之为体积光。

体积光在自然界中是十分常见的现象，如太阳光从云隙中透过时产生的云隙光（图 2-7 左），森林中阳光从树叶中穿过产生的光柱（图 2-7 中），又或是城市中聚光灯在夜晚产生的光影效果（图 2-7 右）。

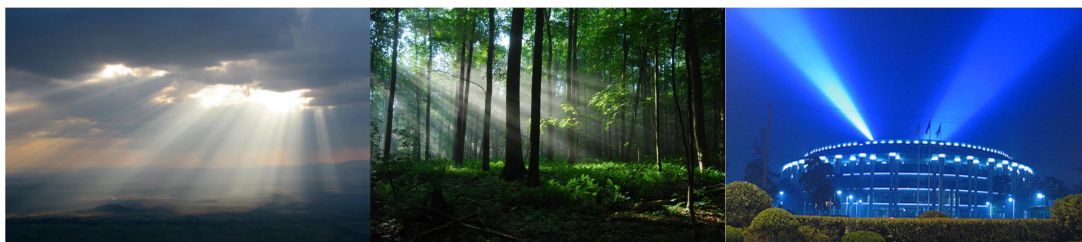


图 2-7 体积光现象

体积光现象有时又被称作“丁达尔效应”，由英国科学家丁达尔在 1869 年发现。其理论基础同样是光线的散射原理，可以使用 2.2.2 小节中所描述的米氏散射理论来解释。对于溶胶，其粒子大小通常在 1-100nm 之间，与可见光的波长相当，所以在光线穿过胶体时，会发生明显的米氏散射现象，产生肉眼可观察到的光柱体。而对溶液来说，由于其粒子大小远小于波长，所以米氏散射效果不明显，也无法观察到丁达尔效应。

云、雾、灰尘等同样也属于胶体，我们称之为气溶胶，其分散剂是空气，而分散质是尘埃或者液滴等。于是当阳光穿过云层或树叶间的空隙时，也会产生肉眼可见的光束，这就是体积光。

为了在计算机中模拟体积光现象，人们提出了许多基于经验的模型。Mitchell 在 2007 年提出了一种简单的在屏幕空间进行射线模糊处理的预计算技术，它的优点是运行

速度很快，但是在背向光源处却无法渲染出正确的效果^[16]。Sousa 在 2008 年使用了一种从场景深度缓存中获取太阳光束混合遮罩的方法，但同样也会出现和前者一样的限制^[26]。

本文所建立的大气模型是基于物理的方法，可以准确的模拟出米氏散射现象，所以当太阳光在遮挡物边缘或空隙中穿过时，同样会产生明显的体积光现象，并且在背向光源处仍然可以渲染出正确的结果。

2.6 本章小结

在本章中，我们介绍了本文的相关理论基础。我们首先阐述了大气散射的概念、原理、作用，并展示了它在计算机虚拟场景中的重要性。然后，我们详细的分析了大气的物理模型，根据空气中不同直径大小的粒子，将散射模型分为瑞利散射和米氏散射两部分，分别讨论它们的原理和散射特点，并解释了大气密度对散射效果的影响。最后，我们对散射积分公式进行了详尽的推导，解释了光学深度、散射系数、相位函数、可见因子等公式中的参数所代表的实际意义，以及它们在算法中的作用。我们展示了该积分公式的复杂性，并分析了对其进行优化的必要性。

第三章 光线散射积分优化算法设计

在第二章中，我们详细的阐述了散射积分的推导过程，并给出了完整的积分计算公式。我们可以看到，散射积分是一个十分复杂的嵌套积分，所以不能直接通过全解析的方式来计算，这并不是指它无法在数学上求解，而是直接求解需要对空间中每个点都进行两个光学深度积分计算，这样的代价在实时渲染中太过昂贵。幸运的是，我们可以使用一些技巧来显著的加快数值积分计算的速度。

3.1 光线步进和梯形法则

首先，我们可以利用积分计算所固有的梯形法则来简化积分的计算。

梯形法则是数学上一种求积分近似解的方法，其原理是将积分计算转化为一个在循环中加权求和的方式。我们可以将整条积分路径分为 n 个采样片段，然后对每个采样片段的中点做积分计算，最后将每个中点的积分结果乘以该片段的长度，再将它们累加起来，就可以得到整条路径的积分值。显然，我们分的采样片段越多，所得到的结果就越精确，但同时也会使得积分的计算量变得更大。

如图 3-1，C 点为摄像机位置，O 点为观察物体的位置，A 点为太阳光入射到大气层时与大气最外层的交点。我们的目的就是为了求从 C 点到 O 点整条路径上的内散射积分，假如我们在 CO 上取 5 个采样点，分别命名为 P1 到 P5，每个点都表示大气中的一个空间点。以 P3 点为例，当太阳光射入大气通过该点时，会发生散射，即从 A 点到 P3 点的路径上会发生外散射衰减，而在 P3 点处，其中一部分光会被散射朝向摄像机，即发生内散射，而这部分内散射光在从 P3 点到达 C 点的路径上，还会再发生外散射衰减。于是，我们可以分别在 P1 到 P5 处计算内散射积分，再乘上每个片段的长度，最后累加即可得到 C 点到 O 点整条路径上的内散射积分。而这一逐步累加计算的过程，我们就称之为光线步进（Ray Marching）。

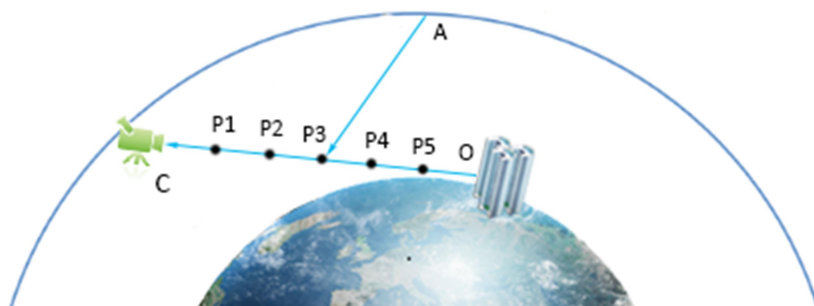


图 3-1 光线步计算散射积分

在实现中, 如果想要得到相对较好的画面质量, 至少需要对从 P 到 C 路径上的内散射积分进行 50 次采样, 对 A 到 P 及 P 到 C 的外散射积分分别进行 5 次采样^[18], 这样, 针对瑞利散射和米氏散射, 需要分别对 3 个 RGB 通道进行两种类型的散射计算, 那么, 就至少要进行 $2*3*50*(5+5)=3000$ 次积分计算^[19]。由于我们必须对每个像素点都进行一次散射积分计算, 而现今的显示器分辨率可以达到 $1920*1080$, 那么, 如果要保证 60 帧的刷新率, 则每秒钟的计算量将达到 $3000*1920*1080*60=373248000000$, 即三千多亿次, 这对于实时渲染来说开销过于巨大了。所以, 如果想在实时交互程序中实现大气散射效果, 就必须要对散射积分的计算进行进一步的优化。

3.2 预计算查找表

对于公式 2-10, 我们可以通过预计算其中一部分积分值, 并存储在查找表中来减少实时计算的开销, 加快渲染速度。

第一步, 我们可以移除对光学深度 $T(A \rightarrow P)$ 的实时计算。O' Neil 发现了公式 2-10 取决于两个参数: 当前点的海拔高度 h 和地表垂直线与光线的夹角 ϕ ^[18]。于是光学深度 $T(A \rightarrow P)$ 就可以先进行预计算, 然后保存在一个查找表中。

首先, 我们重写光学深度函数 (公式 2-9):

$$T(A \rightarrow P) = \beta_R^e e^{-h(t)/H_R} dt + \beta_M^e e^{-h(t)/H_M} dt \quad (3-1)$$

我们可以看到, β_R^e 和 β_M^e 都是三维向量, 而式 3-1 中的两个积分都是标量, 于是我们可以使用一个双通道的查找表来存储瑞利粒子和米氏粒子在大气中从海平面到大气层顶端的密度变化。通过查找表的帮助, 公式 3-1 可以这样来计算:

$$T(A \rightarrow P) = \beta_R^e .xyz \times D[h, \cos(\phi)].x + \beta_M^e .xyz \times D[h, \cos(\phi)].y \quad (3-2)$$

Schuler 曾使用基于查普曼函数的解析公式来对散射积分做近似, 同样可以得到较好的结果^[20], 但是在实现中我们发现这样的做法明显会更加复杂, 而查找表在效率上要快上许多, 所以我们并没有采用他的方法。

而光学深度 $T(P \rightarrow C)$ 的数值积分可以通过跟踪从摄像机到当前点的粒子密度来计算。最后, 我们观察到角度 θ 在射线步进过程中并不进行改变, 所以两个相位函数 $p_R(\theta)$ 和 $p_M(\theta)$ 都可以放到积分外。为了实现这点, 我们对瑞利和米氏内散射分别进行积分, 然后在最后乘上相应的相位函数。

我们计算光学深度积分的过程如表 3-1 中的伪代码所示：

表 3-1 光学深度积分计算过程

```

// 计算光学深度积分的步骤

dP . xyz = (O . xyz - C . xyz) / NSteps ;

ds = ||dP|| ;

// 初始化积分变量

DP→C . xy = 0 ; // 从摄像机位置到积分点的空气净密度

LRlgh . rgb = 0 ; // 瑞利散射

LMie . rgb = 0 ; // 米氏散射

for ( float s = 0.5f ; s < NSteps ; s += 1.0f )
{
    // 计算当前点的位置

    P . xyz = C . xyz + dP . xyz * s ;

    // 计算当前海拔和地表的法线

    h = || P - CEarth || - REarth ;

    NEarth . xyz = ( P . xyz - CEarth . xyz ) / || P . xyz - CEarth . xyz || ;

    // 计算 P 点处的瑞利和米氏粒子密度缩放系数

    ρRM . xy = e-h/HRM . xy ;

    // 计算 cos(φ) 并查找从 P 点到大气层穹顶的粒子密度

    cosφ = ( NEarth - 1 ) ; // 1 是光线的方向

    DA→P . xy = D[h, cosφ] ;

    // 累加摄像机位置的粒子密度

    DP→C . xy += ρRM . xy * ds ;

    // 计算从大气层穹顶经过积分点到达摄像机的总粒子密度

    DA→P→C . xy = DA→P . xy + DP→C . xy ;

    // 计算瑞利散射和米氏散射的光学深度

    TR . xyz = DA→P→C . x * βRe . xyz ;

    TM . xyz = DA→P→C . y * βMe . xyz ;

```

```

// 计算当前积分点位置的衰减
 $E_{R+M} \cdot \text{rgb} = e^{-(T_R \cdot \text{xyz} + T_M \cdot \text{xyz})}$ ;

// 计算内散射的差值
 $dL_{Rlgh} \cdot \text{rgb} = \rho_{RM} \cdot x \cdot \beta_R^s \cdot \text{rgb} * E_{R+M} \cdot \text{rgb} * ds$ ;
 $dL_{Mic} \cdot \text{rgb} = \rho_{RM} \cdot y \cdot \beta_M^s \cdot \text{rgb} * E_{R+M} \cdot \text{rgb} * ds$ ;

// 计算可见因子 V
// 更新瑞利散射和米氏散射
 $L_{Rlgh} \cdot \text{rgb} += dL_{Rlgh} \cdot \text{rgb} * V$ ;
 $L_{Mic} \cdot \text{rgb} += dL_{Mic} \cdot \text{rgb} * V$ ;
}

// 乘以米氏散射和瑞利散射的相位函数
 $\cos\theta = (v, l)$ ; // v 是视线方向
ApplyPhaseFunctions( $L_{Rlgh} \cdot \text{xyz}$ ,  $L_{Mic} \cdot \text{xyz}$ ,  $\cos\theta$ );

// 计算从摄像机位置开始的内散射和衰减
 $L_{In} \cdot \text{rgb} = (L_{Rlgh} \cdot \text{rgb} + L_{Mic} \cdot \text{rgb}) * L_{Sun}$ ;
 $Ext \cdot \text{rgb} = e^{-(DP \rightarrow C \cdot x \cdot \beta_{eR} \cdot \text{xyz} + DP \rightarrow C \cdot y \cdot \beta_{eM} \cdot \text{xyz})}$ ;

```

3.3 极线采样

在 3.2 节中,我们通过一张预计算的查找表来加速光学深度积分 $T(A \rightarrow P)$ 的计算过程,但 $T(P \rightarrow C)$ 仍需要对屏幕上的每个像素点都做实时计算(对于 1080P/60FPS 来说就是每秒 1 亿 2000 万次),代价依然太过昂贵。因此,我们必须对这样的计算过程做进一步的优化。

对内散射积分计算的优化,人们做过许多的研究和尝试(见 1.2 节)。其中 Engelhardt 和 Dachsbacher 在 2010 年提出了一种优雅而高效的渲染光线散射的算法,他们发现了散射光的辐射度是沿着光源在屏幕上的投影点发出的极线均匀变化的^[23],而这样现象可以通过极几何来很容易的表示^[27]。于是我们就可以只对屏幕上某些标记的采样点做散射积分计算,而其余点的积分值可以通过插值得到。

针对内散射光这样的特点,我们可以通过以下的步骤生成采样点:

- (1) 在屏幕边缘放置可定义数量的样本点,这些样本点是等距的。

(2) 将光源位置在屏幕空间上的投影点与这些样本点连接起来，这些放射状的线段就是我们所需要的极线。

(3) 在这些极线上放置等距的样本点，数量同样是可定义的。

(4) 在深度不连续的点前后添加额外的样本，以保证插值结果的正确性。

(5) 对所有的样本点做基于光线步进的内散射积分运算。

(6) 沿着极线片段做插值，然后在极线片段之间做插值，从而计算出屏幕剩余所有像素点的内散射积分值。

我们将对每个步骤的实现细节进行详细的解释。

3.3.1 极线生成

该算法的关键在于如何沿着极线放置样本，我们可以分别对如下几种情况进行讨论。

我们首先假设光源处于摄像机的正前方，且光源位置可以投影到屏幕范围内。我们就可以将光源在屏幕上的投影点与屏幕边缘可定义数量的出口点连接起来，这样就很自然的生成了我们所需要的极线，如图 3-2。

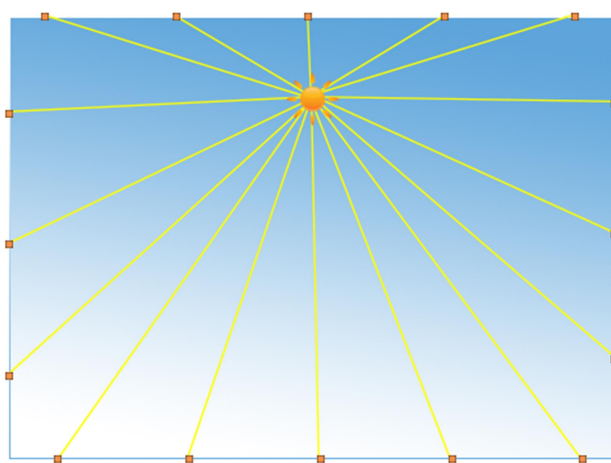


图 3-2 光源投影点在屏幕内的极线生成

而当光源的投影位置在屏幕外时，有一些极线会完全处于可见范围外，则可以将这些不可见的极线完全剔除，避免在接下来的步骤中再做额外的计算。如图 3-3，红色的线段是我们需要剔除的不可见极线。

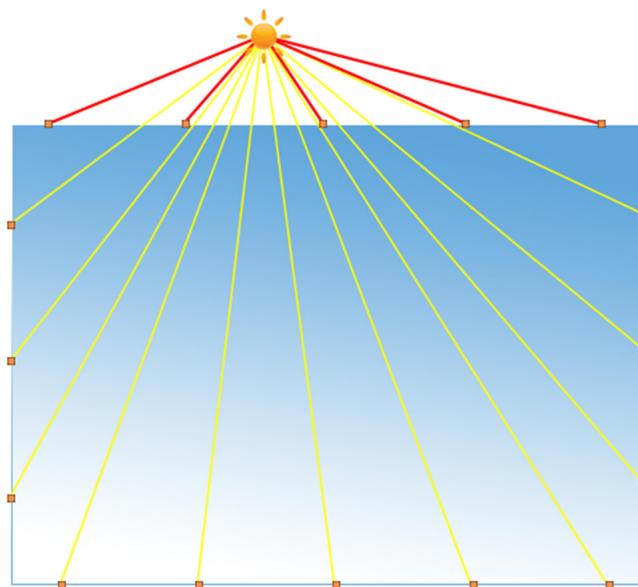


图 3-3 光源投影点在屏幕外的极线生成

如果光源在摄像机后，那么它的投影位置将是光线到无穷远处的汇聚点，样本生成的方法是一样的。

如果摄像机位置和光源位置正好完全重合（这是一种很少见的情况），那么极线将退化成点，极线采样算法也将退化为简单的屏幕空间子采样。

3.3.2 样本生成

在生成了极线后，我们就可以将可定义数量的初始样本点均匀的放置在这些极线上，通常 8 到 32 个初始样本点已经足够表现出内散射光的变化特征^[23]。这些样本点将极线切割为极线片段，如图 3-4 所示（为展示起见，此处每条极线上只有 4 个初始样本）。

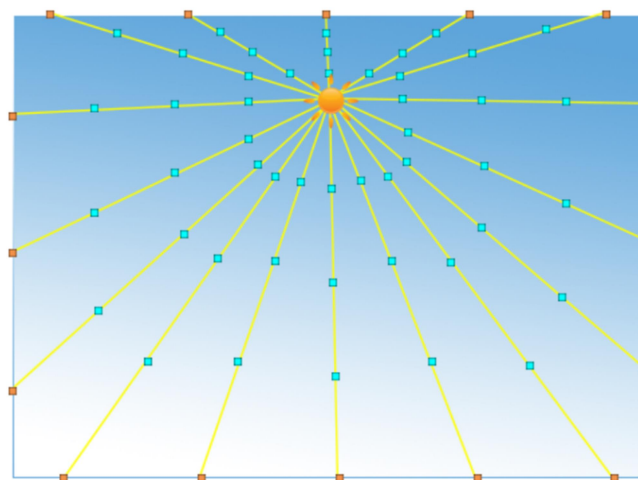


图 3-4 在极线上生成初始样本点

如果光源的投影位置和屏幕边缘很近，那么不同极线的长度将会差别很大。对短的极线来说，可能会导致样本点分布太过密集而做冗余的计算。

为了解决这个问题，我们通过延长边界点来重新调节极线，尽量使所有极线上的样本点都维持相同的间隔。这种做法不仅减少了大量的计算，并且可以产生更加自然的环形分布结果，而不是 Engelhardt 等人的算法中所得出的矩型分布结果^[25]。

3.3.3 样本细化

在生成了极线和初始样本点后，我们需要检测极线片段中的深度不连续点。这是由于内散射辐射度可能会在深度不连续点处突然变化，所以我们必须在间断点前后放置额外的样本点以保证插值结果的正确性。如图 3-5，红色的点为我们在 3.3.2 节中生成的初始光线步进样本点，蓝色的点是根据插值计算额外添加的样本点，黑色的线表示场景深度在该处出现不连续的变化，绿色的点即为我们在间断点处额外放置的样本点。

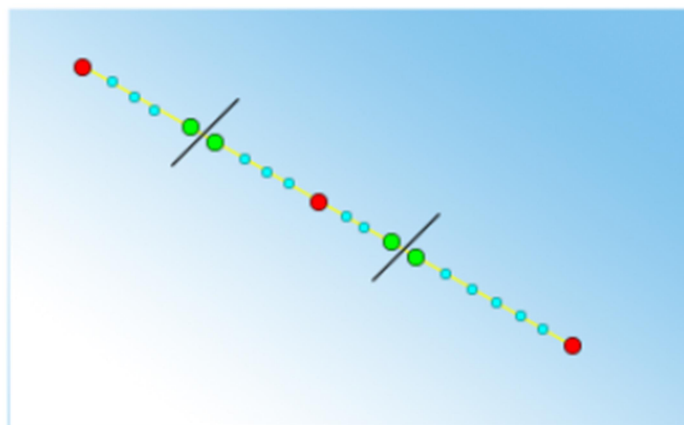


图 3-5 在深度间断点处做样本细化

Engelhardt 等人的做法较为简单，他们直接沿着极线来线性查找场景的深度缓存，以找到深度数据骤变的点^[23]，但这样的做法需要对阴影贴图上的纹素进行遍历，开销同样很大，尤其是对于高分辨率的贴图来说。而使用常量采样的方法则会引起采样过疏而导致人工痕迹的出现。为此，我们可以通过构建一维最小/最大二叉树来加速这一计算的过程^[24]。具体方法将在 3.4 节中进行讨论。

3.3.4 可见因子与阴影贴图

在我们对散射积分做光线步进计算时，还需要考虑到阴影的影响。当光线通过被阴影遮挡的区域时，我们认为该部分的片段不会对内散射光造成影响，所以在计算中可以

将这些片段剔除。我们通过在散射积分计算中设置可见因子 $V(P)$ 来达到检测和剔除的目的（公式 2-10）。在做光线步进时，对于被太阳光完全照亮的区域，我们将可见因子 $V(P)$ 设为 1，而对于完全处于阴影中的区域，我们将其可见因子 $V(P)$ 设为 0。

在我们的光线步进计算散射积分的算法中，对可见因子的检测是在场景的阴影贴图（Shadow Map）中进行的，所以在正式介绍我们的光线步进算法之前，我们首先介绍一下什么是阴影贴图（Shadow Map）。

阴影贴图是计算机图形学中渲染阴影效果的一种常用的技术，由 Lance Williams 在 1978 年首次提出^[28]，由于它的高效性，很快的被广泛应用于各种实时和离线渲染应用中。

阴影贴图技术利用了一个很简单的原理，即从光源位置看去，所有可以看到的物体会被光源照亮，所有被遮挡不可见的物体都处于阴影中。那么，我们可以先从光源位置将场景渲染到一张贴图中，但只渲染深度信息，即该贴图中保存的是离光源位置最近点的距离。然后，在从视点处渲染场景时，计算场景中每个可见点到光源的距离，并与保存的深度贴图信息做比较：如果距离等于深度，则说明该点离光源点最近，被光源照亮；如果距离大于深度，则说明该点被遮挡，位于阴影中。如图 3-6 所示。

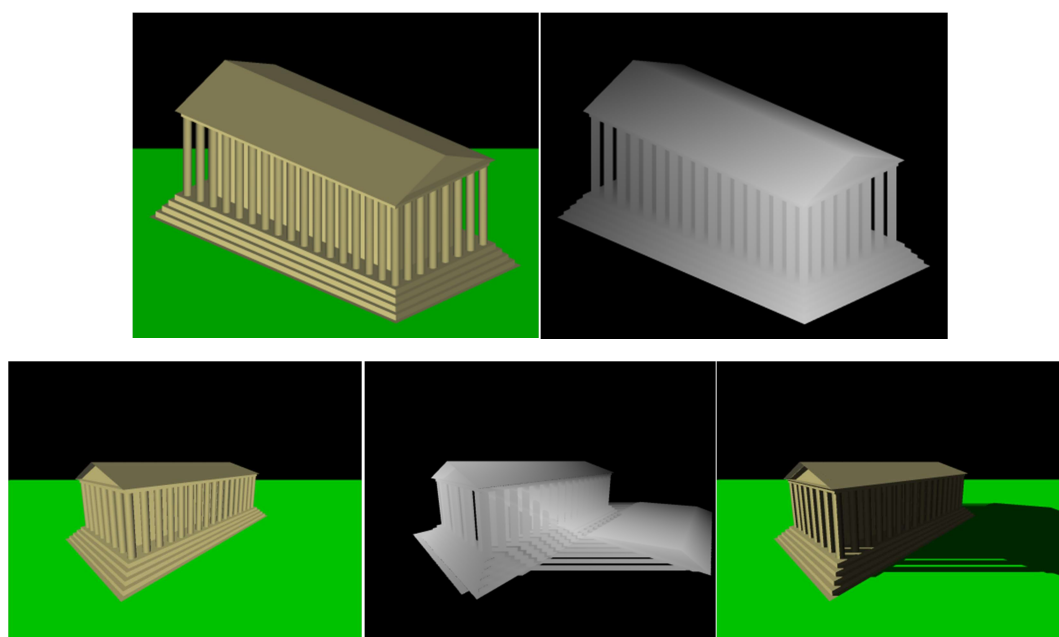


图 3-6 阴影贴图

由于在现代图形硬件中，渲染深度信息是一个非常快速的过程，所以使用阴影贴图技术来渲染阴影，具有很高的效率。而在计算散射积分时，由于需要考虑到阴影的影响，所以为了确定可见因子 $V(P)$ ，所以我们需要在阴影贴图空间中做光线步进计算。

3.3.5 光线步进

在生成了样本点后，我们对每个样本点做基于光线步进的內散射积分计算。对于每个样本点，我们都发射一条从摄像机位置开始的射线，然后将它的起点和终点变换到以光源为视点的阴影贴图空间，最后在阴影贴图空间中做光线步进计算，如图 3-7。

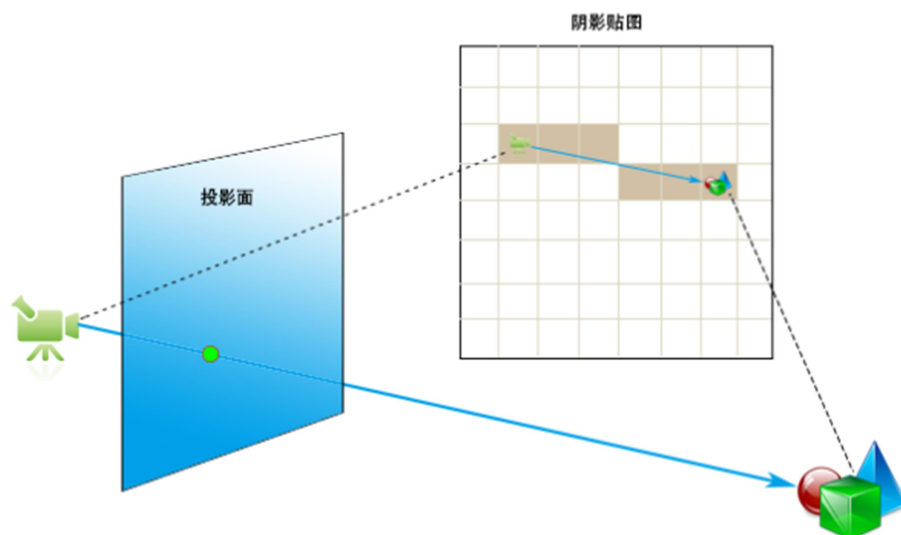


图 3-7 在阴影贴图空间中做光线步进

对于每一个样本点，我们都做如下步骤的处理：

- (1) 将从样本点到观察物的视线投影到阴影贴图中。
- (2) 沿着极线遍历阴影贴图，对每一个阴影贴图纹素，计算从当前点到无穷远处的內散射积分值。
- (3) 该片段的光线散射总和是上一个纹素点的积分与当前纹素点积分的差，再乘以可见因子，最后将它累加起来。
- (4) 继续计算下一个纹素点，直到对整条光线做完光线步进计算。

像这样的从屏幕点开始，对每条光线做逐像素计算并累加所得结果直到遇到物体为止的操作，就叫作光线步进。光线步进算法可以生成很好的视觉结果，但如上的简单光线步进算法必须对阴影贴图中每个纹素点都执行计算。即使在高性能的 GPU 上，这样的实现也需要花费很长的时间，所以需要进行进一步的优化。

3.4 一维最小/最大二叉树

3.4.1 极切片

从同一条极线样本点上发射出的摄像机射线，显然是处于同一平面内，该平面我们

称之为极切片，如图 3-8。

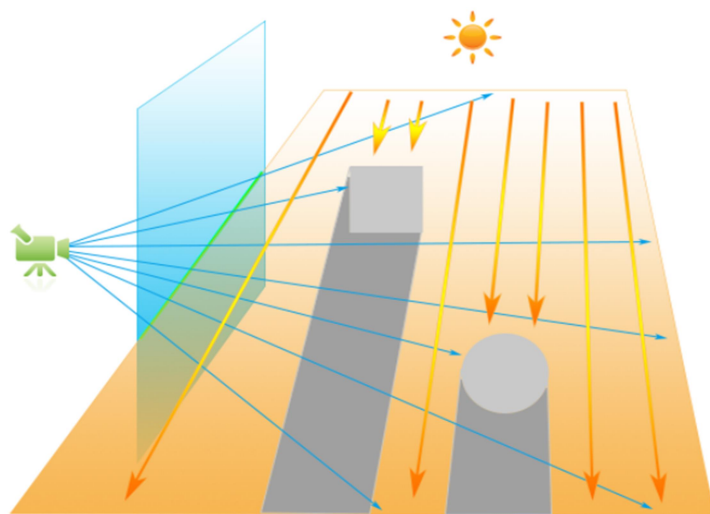


图 3-8 极切片

由于极切片与光线投影面是垂直的，所以二者的交线是一条直线。极切片中所有的摄像机射线都会投影到这条直线上，并且只是终点是不同的。

在这条直线上做阴影贴图采样的结果就是一张一维的高度图，这张高度图适用于该极切片上的每条射线。为了计算等式 2-10 中的可见因子 $V(P)$ ，我们只需要判断视线的当前位置是在高度图的上方还是下方即可，如图 3-9。

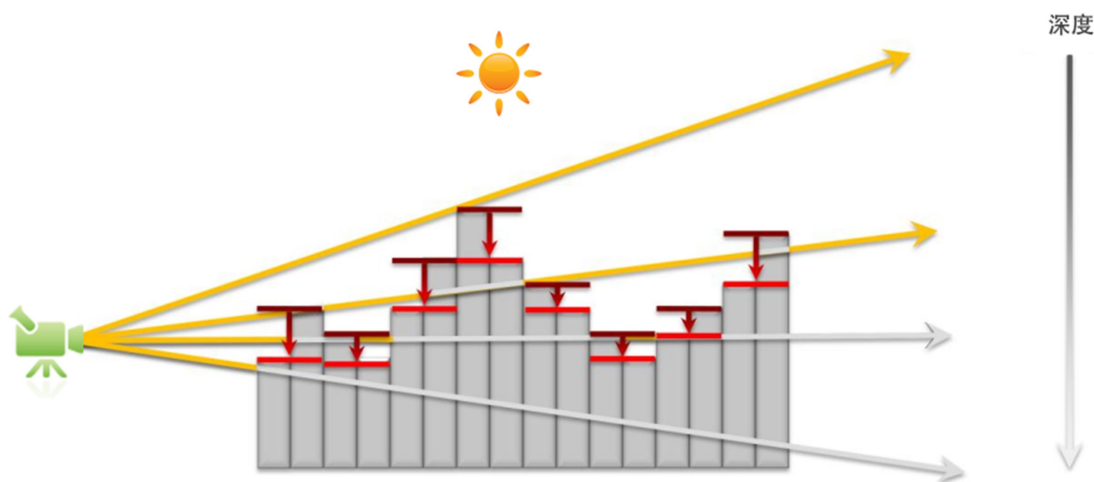


图 3-9 通过极切片生成的一维高度图

这样一来，我们就可以很清楚的找到射线上被照亮或被阴影遮挡的片段，而不用对所有潜在的阴影贴图样本做计算了。为了更快速的检测这样的片段，我们使用 Chen 等人提出的一维最小/最大二叉树数据结构^[24]，如果射线片段的最大深度值比树结点中存储的最小深度值还要小，那么该片段就是被完全照亮的。反之，如果最小深度值比结点中

的最大深度还要大，那么该片段就是完全处于阴影的。

3.4.2 创建最小/最大二叉树

要为极切片创建一维最小/最大二叉树，我们首先需要定义切片和阴影贴图的交线。由于切片上所有的摄像机射线都会被投影到这条交线上，那么我们就可以使用射线上的任意两个点并把它们投影到阴影贴图 UV 空间中，然后再连接这两个投影点，即可得到所求的交线。

在本文的实现中，我们使用摄像机位置作为第一个点，它在 UV 平面的投影位置 O_{UV} 作为我们在高度图上的第 0 个样本。而终点我们选择为极线在屏幕的边缘出口点的射线终点，将它变换到 UV 空间然后通过 O_{UV} 计算朝向 D_{UV} 。然后，我们将 D_{UV} 标准化，使它在 U 或 V 轴上的投影最大长度为 1。如果 O_{UV} 在阴影贴图边界 $[0, 1] \times [0, 1]$ 之外，我们就沿着 D_{UV} 方向，将源点移动到与边界的第一个交点处，如图 3-10 右。

现在，当我们知道了一维高度图中的第 0 个样本 O_{UV} 和朝向 D_{UV} ，我们就可以设定第 i 个样本为 $O_{UV} + i \cdot D_{UV}$ 。最底层的树结点可以通过每 $2i$ 个和 $2i+1$ 个样本来构建，更上层的树结点都可以用这些值进行再计算。而在图 3-10 左中，原始样本在阴影贴图的位置并不在像素中央，于是为了得到正确的结果，我们通过对最近的四个样本做 PCF 滤波（Percentage Closer Filtering，百分比逼近滤波，是一种常用的柔化阴影贴图边缘，从而产生软阴影的技术^[30]）来计算保守的最小/最大边界，详见 4.4 节。

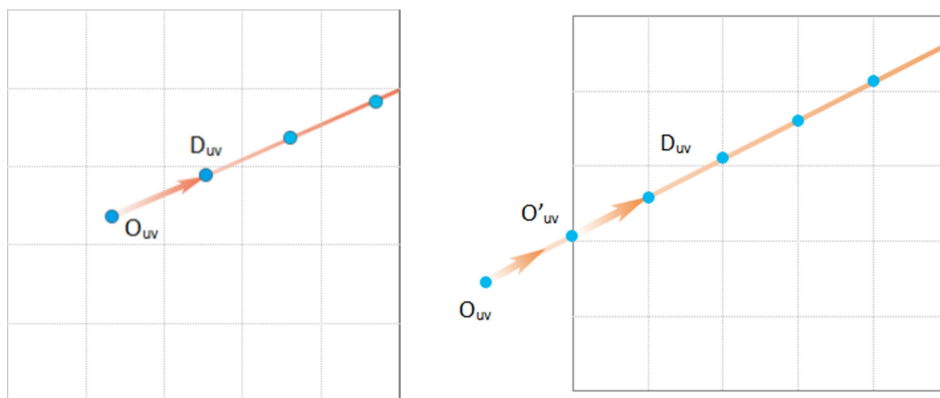


图 3-10 极切片的源点和朝向

如图 3-11，阴影贴图的每一行都表示一个极切片的一维高度图，我们希望快速的找到高度图和摄像机射线的交点（图 3-11 下）。为此，我们为每行计算一个满二叉树，该树的每个结点都存储着高度图在该结点下方的深度最大值和最小值（图 3-11 上）。然后

我们将两两结点进行合并，得到更上层的结点，其中存储的是以该结点为子树的所有叶子结点的最大值和最小值。重复以上步骤，直到生成了根结点为止。最后，我们就可以得到一棵满二叉树，我们称之为二维最小/最大二叉树，使用该数据结构可以不需要进行循环就可以进行遍历。

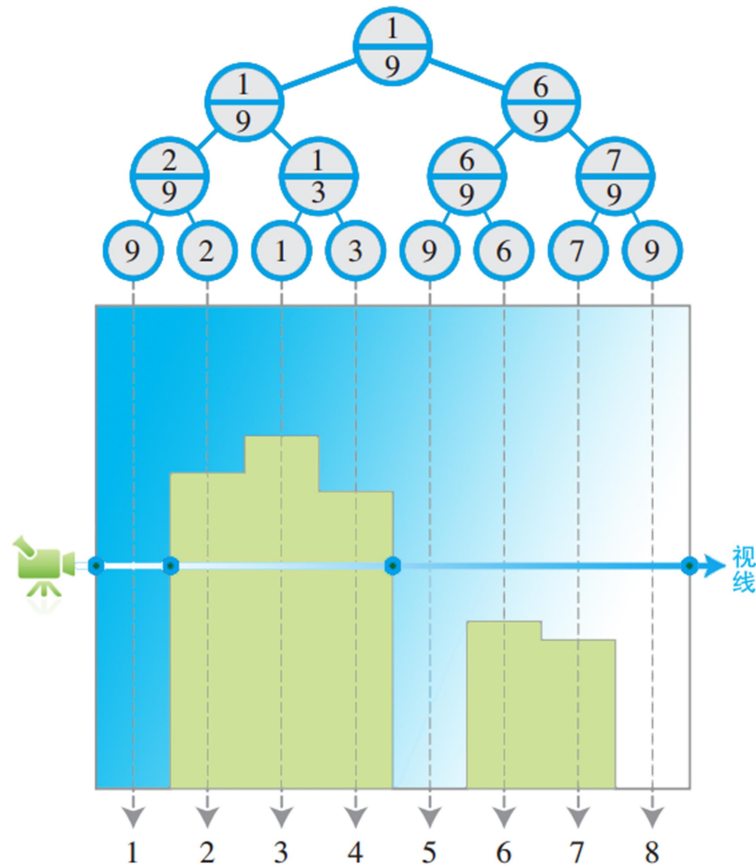


图 3-11 一维最小/最大二叉树

3.4.3 使用一维最小/最大二叉树检测可见因子

在建立了一维最小/最大二叉树之后，我们就可以使用这一数据结构来进行光线步进计算。本文采用 Tevs 等人在 2008 提出的光线步进优化算法^[29]，该算法同样被 Chen 等人在他们的实现中利用^[24]。在该算法中，最小/最大树的遍历不需要进行循环就能实现。

对每条摄像机射线，我们可以使用一个递归算法来遍历它，而在本文的像素着色器实现中进一步的避免了递归调用。假设我们在 β 高度有一条摄像机射线，然后我们从树的根结点开始计算。对于当前树结点时，如果 β 比结点中存储的最小值还要小，那么该结点就是完全被照亮的，我们就对该结点表示的范围做积分计算。如果 β 比结点中存储的最大值还要大，该结点表示的范围就是完全在阴影中，我们就跳过它。否则的话，我

们就递归的处理两个孩子结点。而为了避免递归调用，当我们在计算一个结点时，我们并不弹出调用栈，而是通过索引来顺序遍历到下一个结点，该结点的下一个索引位置是它的左孩子或者是它的祖先的一个右结点。这一过程可以在 GPU 上高效的实现，其着色器的部分关键伪代码如表 3-2 所示。

表 3-2 一维最小/最大二维树的遍历过程

```

float depth = D[x, y]
float ( $\alpha$ ,  $\beta$ ,  $\gamma_1$ ) = TO-EPIPOLAR(x, y, depth)
int node = 1 // 从根结点开始
int2 ( $\gamma_-$ ,  $\gamma_+$ ) = RANGE(node)
float4 out = (0,0,0,0)
do
if min[node,  $\alpha$ ]  $\leq \beta <$  max[node,  $\alpha$ ]
    node = node * 2 // 递归左子树
else // 完全被照亮或者完全处于阴影中
    if  $\beta <$  min[node,  $\alpha$ ] // 完全被照亮的节点
        out = out +  $\Gamma_{\text{sum}}[\min(\gamma_1, \gamma_+)] - \Gamma_{\text{sum}}[\gamma_-]$ 
    end if
    node = node + 1 // 继续遍历下一节点
    while node is even // 若该结点是个左孩子
        node = node / 2 // 回到父结点
    end while
end if
( $\gamma_-$ ,  $\gamma_+$ ) = RANGE(node)
while node  $\neq$  1 &&  $\gamma_1 > \gamma_-$  // 当射线穿过该结点时
return dot(B[ $\beta$ ], out)

```

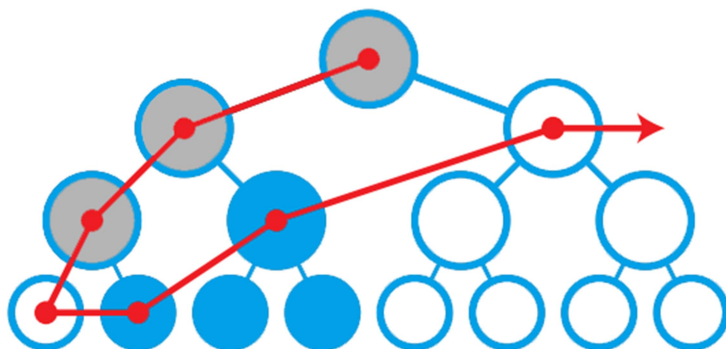


图 3-12 一维最小/最大二叉树的遍历

以图 3-12 为例，图中的白色结点表示其范围内是完全被照亮的，蓝色结点表示其范围内是完全处于阴影中的，而灰色结点表示其范围内部分被照亮，而部分处于阴影中，红色的箭头表示我们遍历的顺序。假设我们需要对高度为 5 的摄像机射线做光线步进检测，则从根结点开始，根结点表示的高度范围为 1-9，而 5 处于该范围中，所以需要进一步对其子结点做检测。将其标志为灰色，再向左下遍历，直到叶子节点。9 大于 5，表示它是被照亮的，则将其标为白色，2 小于 5，则将其标为蓝色。再通过索引到祖先结点的右子树，由于 1-3 的最大值 3 小于 5，则说明该子树的所有范围都处于阴影中，将它们都标记为蓝色。回到根结点的右子树，由于 6-9 的最小值大于 5，说明该子树的所有范围都处于光线的直射中，则将它们都标记为白色。至此，则完成了二叉树的遍历，并得到了这条射线的所有片段的 $V(P)$ 因子。

3.4.4 像素插值

在对屏幕上每一个采样点完成了基于光线步进的散射积分计算之后，我们需要对屏幕上剩余的像素点做插值计算，以得到最终渲染的图像。Engelhardt 和 Dachsbacher 发现了一种双边滤波器，能从极线中直接画出滤波信号，可以很好的适用于光线步进样本并同时保持边缘特征^[23]。

为了对屏幕上的每个像素点计算内散射辐射度，我们首先需要找出离该像素点最近的两条极线，然后沿着极线进行插值。首先，我们将像素位置（图 3-13 中的红色位置）垂直投影到最近的两条极线上，并将两个投影点的距离定义为 1。然后，我们在两条极线上离投影点距离为 1 的四个点处做插值计算（图 3-13 中的绿色点），而这些投影点处的内散射光强度我们使用光线步进样本做线性插值。在分别对每条极线上的投影点都产生了各自的插值颜色后，我们对这些插值进行权值追踪，根据深度信息计算它们的加权

和，最后根据权值对两个颜色进行插值，得到最终的像素颜色结果。

然而，也可能出现在极线上找不到合适的样本点的情况（所有插值点的权值都接近零），这种情况在场景沿着极线出现深度不连续时尤其明显。在这些情况下我们使用各自的更靠外的两条极线，以同样的方法来获取合适的插值点，再重复上述的操作。

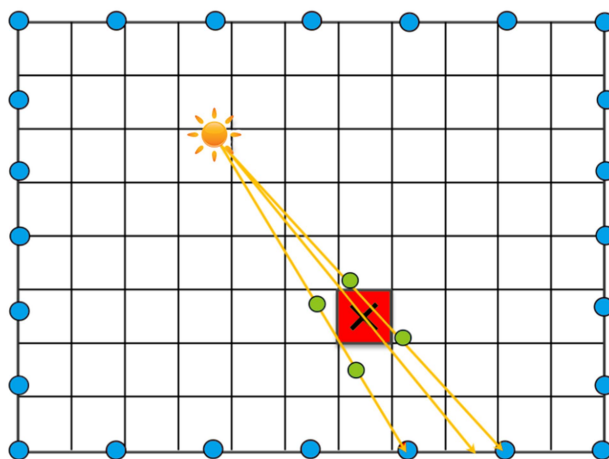


图 3-13 像素插值

上述的插值算法虽然简单，但在大部分时候都可以得到令人满意的视觉结果。当然，任何这样的插值算法都会在某些特定的场景环境下丢失信息。Wyman 和 Ramsey 曾尝试在插值点的深度值与目标像素点相当、明显大于像素点、或明显小于像素点的三种情况下分别使用不同的滤波方法，可以得到相似的插值结果，但却增加了计算的开销^[31]。

3.5 本章小结

在本章中，我们详细的阐述了我们的光线散射积分优化算法的思想和设计。首先我们介绍了我们计算散射积分的核心方法——利用积分的梯形法则进行光线步进计算。然后，我们分别阐述了我们如何一步一步的对该计算过程进行优化：使用一张预计算的二维查找表来简化散射积分运算，使用极线采样的思想来降低样本点的数量，使用一维最小/最大二叉树的数据结构来加速可见因子的计算。并详细解释了每一步优化算法的核心思想和计算过程。

第四章 散射积分优化算法的 GPU 实现

4.1 DirectX 11 介绍

4.1.1 DirectX 简介

DirectX（简称 DX）是由微软提出的一套针对多媒体编程和游戏开发的应用程序接口（API），可以并仅可以用于微软的软硬件平台上，如 Windows、XBOX 360、XBOX ONE 等。最初的 API 命名都是以 Direct 开头的，如 Direct3D, DirectDraw, DirectMusic, DirectPlay, DirectSound 等，所以使用 DirectX 来做所有这些 API 的总称。当微软开始制作游戏主机后，X 也被用作 XBOX 的前缀，以表明该主机是基于 DirectX 技术开发的。

Direct3D（DX 中包含的 3D 图形 API）被广泛应用于 Windows 和 XBOX 上的视频游戏开发，同时也被其它软件如 CAD/CAM 工程用于可视化和图形渲染。由于 Direct3D 是 DirectX 中最广泛使用的组件，有时候这两个名词也可以进行通用。

4.1.2 DirectX 的组件构成

Direct3D（D3D）：用于绘制三维图形。

DXGI：用于枚举显示适配器和显示器，并管理 Direct3D 10 后的缓存交换链。

Direct2D：用于绘制二维图形。

DirectWrite：用于绘制文字。

DirectCompute：用于 GPU 通用计算。

DirectX Diagnostics (DxDiag)：一个对 DirectX 中的其他组件做诊断和生成报告的工具。

DirectX Media Objects：提供对流对象的支持，如编码和解码等。

DirectSetup：用于 DirectX 组件的安装，并检测当前的 DirectX 版本。

XACT3：高级别的音频 API。

XAudio2：低级别的音频 API。

4.1.3 Direct3D

Direct3D 是一个图形编程接口，是 DirectX 的一个组件。Direct3D 多用于在实时性能要求较高的程序（如游戏）中渲染三维图形。当显卡允许时，Direct3D 会使用显卡所支持的硬件加速技术，允许对整个三维渲染流水线进行硬件加速，或者只加速其中的某个部分。Direct3D 公开了它的先进的图形处理能力，包括 Z 缓冲（Z-buffering）、W 缓冲

(W-buffering)、模板缓冲 (Stencil buffering)、空间反走样 (spatial anti-aliasing)、 α 混合 (alpha blending)、颜色混合 (colour blending)、Mip 映射 (mipmapping)、纹理混合 (texture blending)、裁剪 (clipping)、剔除 (Culling)、大气效应 (atmospheric effects)、透视校正纹理映射 (perspective-correct texture mapping)、可编程的 HLSL 着色器 (programmable HLSL shaders) 等。与 DirectX 的其它技术集成使得 Direct3D 可以提供如全景视频、基于硬件的三渲二技术、点精灵等功能, 实现了在交互式媒体中的二维和三维图形渲染。

Direct3D 包括了许多三维计算机图形渲染的命令, 然而从第 8 版开始, Direct3D 取代了 DirectDraw 框架, 并接管了二维图形的渲染。微软努力的不断更新 Direct3D 以支持最新的三维图形显卡所提供的新技术。Direct3D 对一些硬件不支持的功能提供了全顶点的软件模拟, 但并不支持全像素的软件模拟。例如, 如果软件使用了像素着色器但用户的显卡却不支持该技术, Direct3D 并不会模拟它, 尽管它仍会计算和渲染三维模型的多边形及纹理, 但相对于硬件实现来说通常会在效果和性能上都会低的多。Direct3D 同时拥有一个参考光栅器 (Reference Rasterizer), 可以用于模拟通用的图形显卡, 尽管它对于大多数的实时三维应用都太过缓慢, 所以通常只作调试使用。Direct3D 10.1 版本实现了一个新的软件光栅器 WARP, 它的性能在多核 CPU 上可以和低端的三维图形显卡媲美。

作为 DirectX 的一部分, Direct3D 在 Windows 95 及以上的操作系统都可以使用, 并且是 XBOX 360 及 XBOX ONE 控制台系统的图形 API 基础。Direct3D 的主要竞争对手是 Khronos 小组的 OpenGL, 以及它的后续版本 Vulkan。微软和 SGI 在 90 年代曾提出了 Fahrenheit, 用于整合 OpenGL 和 Direct3D, 但最终并没能实现。

4.1.4 DirectX 11 的新特性

相对于 DirectX 10 来说, DirectX 11 提供了很多新的特性, 而二者最大的不同之处在于渲染管线。DirectX 11 渲染管线的新特性, 从一定程度上标志着图形硬件和软件功能革命性的进步, 其中很重要的一点是 DirectX 11 加入了对镶嵌着色器 (Tessellation Shader) 的支持。镶嵌着色器由镶嵌单元 (tessellator)、外壳着色器 (Hull Shader)、以及域着色器 (Domain Shader) 组成。另外, DirectX 11 还加入了计算着色器 (Compute Shader), Compute Shader 和 DirectX 10 中引入的几何着色器 (Geometry Shader) 不同, 它并属于渲染管道的一部分, 而是一个可选项。Compute Shader 同样是 DirectX 11 的重要新特性

之一，提供了使用 GPU 进行通用计算的可能，而对于图形程序员来说，也提供了极大的方便和灵活性。DirectX 11 的渲染管线和新特性与图 4-1 所示，其中绿色部分为 DX11 的新功能。

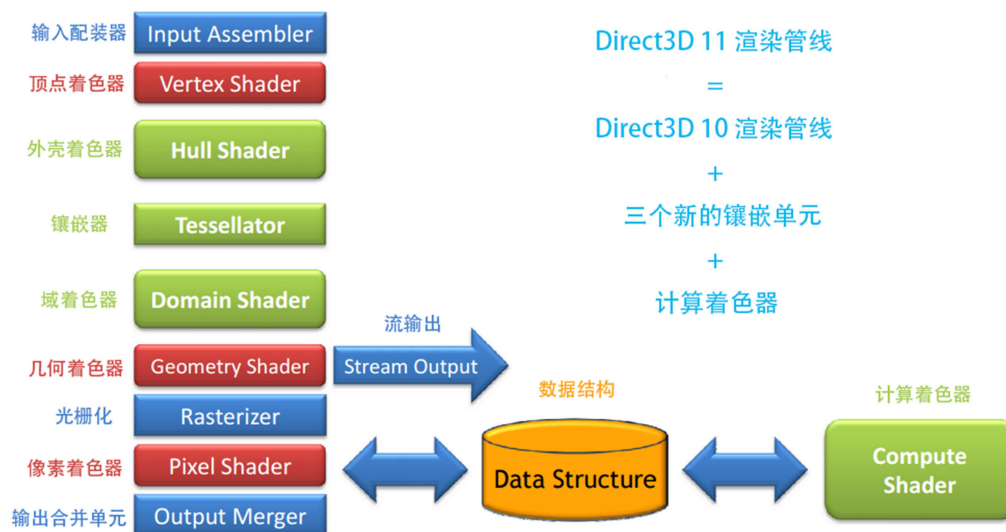


图 4-1 DirectX 11 的新特性

4.2 算法整体流程

在本节中我们将详细介绍算法在实现中的具体步骤，我们的方法基于现代的图形硬件设计并使用 Direct3D 11 来实现。最终图像的渲染分为 3 个步骤：（1）将只有直接光照影响的场景渲染到一个离线缓存中；（2）使用预计算查找表、极线采样、一维最小最大二叉树、像素插值来计算整张个场景的内散射光分布；（3）最后再加上衰减后的物体反射光影响，得到最终的像素颜色。

算法在 Direct3D11 上实现的整体流程如下：

1. 通过逆转深度缓存中的深度值 `tex2DDepthBuffer` (大小为 $W_{Scr} \times H_{Scr}$, 格式为 D32F), 将摄像机空间的 z 坐标渲染到一个屏幕大小的纹理 `tex2DCamSpaceZ` (大小为 $W_{Scr} \times H_{Scr}$, 格式为 R32F) 中。
2. 计算每条极线终点的屏幕坐标, 然后将它们渲染到一个大小为 $N_{Slices} \times 1$ 的辅助纹理 `tex2DSliceEndPoints` (格式为 RGBA32F) 中。
3. 渲染坐标系纹理 `tex2DCoordinates` (大小为 $N_{Samples} \times N_{Slices}$, 格式为 RG32F)、极坐标系下的摄像机空间 z 纹理 `tex2DEpipolarCamSpaceZ` (大小为 $N_{Samples} \times N_{Slices}$, 格式为 R32F), 并设置模板缓存 `tex2DEpipolarStencil` (大小为 $N_{Samples} \times N_{Slices}$, 格式为 D24US8)。

➤ 极线上每个样本点的屏幕坐标都使用从 `tex2DSliceEndPoints` 纹理中加载的极线终点的坐标进行插值计算得出（如 3.3 节所述）。

➤ 为了计算样本的摄像机空间坐标 z ，在样本位置对 `tex2DCamSpaceZ` 纹理做线性滤波。

➤ 设置模板缓存 `tex2DEpipolarStencil`，用于标记出在屏幕上的样本点，设置增量模板函数并剔除在屏幕外的样本点。

4. 对每个可用的极线样本，计算 3.3 节中所描述的粗略内散射积分。然后存储到 `tex2DInitialInsctr` 纹理中。

➤ 模板 `tex2DEpipolarStencil` 用于剔除所有不可用的样本。

➤ $N_{\text{steps}}=7$ 的梯形积分提供了一个在效果和性能间很好的平衡。

➤ 在这一步我们同时会渲染一个衰减纹理 `tex2DEpipolarExtinction`（大小为 $N_{\text{Samples}} \times N_{\text{Slices}}$ ，格式为 `RGBA8U`），用于柔化背景。

5. 下一步，使用存储在 `tex2DInitialInsctr` 纹理中的粗略内散射积分细化样本并计算插值源纹理 `tex2DInterpolationSource`（大小为 $N_{\text{Samples}} \times N_{\text{Slices}}$ ，格式为 `RG16U`）。该过程在 4.3 节中将会有详细描述。

6. 之后，对 3.4.1 小节中所描述的每个极切片定义切片起点和朝向，并将它们存储在另一个辅助纹理 `tex2DSliceUVDirAndOrigin`（大小为 $N_{\text{Slices}} \times N_{\text{Cascades}}$ ，格式为 `RGBA32F`）中。

7. 然后，构建 3.4.2 小节中所描述的最小/最大二叉树，并将其存储在 `tex2DMinMaxDepth`（大小为 $SM_{\text{Dim}} \times (N_{\text{Slices}} \cdot N_{\text{Cascades}})$ ，格式为 `RG16U`）纹理中，该过程将在 4.4 节中详细介绍。

➤ `tex2DSliceUVDirAndOrigin` 纹理用于获取第 0 个样本点位置和切片朝向，通过它计算所需要的样本点位置，并构建二叉树的第一层（最深层）。

➤ 随后根据 3.4 节中所描述的算法构建二叉树更上方的粗略层。

8. 下一步，使用插值源纹理 `tex2DInterpolationSource` 更新模板 `tex2DEpipolarStencil` 并标记出需要执行光线步进样本。注意，所有已被剔除的样本不需要再进行处理。

9. 对所有已标记的样本做光线步进计算。将得到的内散射积分值渲染到 `tex2DInitialInsctr` 纹理中（注意该纹理此前被用于存储初始内散射积分，而在该步骤中又

被重用了)。该过程的细节将在 4.5 节中介绍。

10. 下一步, 使用插值源纹理 `tex2DInterpolationSource` 将初始的内散射光积分值 `tex2DInitialInsctr` 插值到所有的极线样本中, 并渲染到 `tex2DEpipolarInscattering` 纹理中。

11. 最后, 将从 `tex2DEpipolarInscattering` 纹理中得到的内散射光强从极坐标空间转换到笛卡尔坐标空间, 进行像素插值计算, 并加入衰减的物体反射光作用。该过程将在 4.6 节中进行详细介绍。

4.3 极线采样

极线采样和内散射光的计算步骤如下:

- (1) 确定极线的分布并放置初始样本。
- (2) 检测极线上所有的深度不连续点, 以确定哪些样本点必须使用光线步进计算, 而哪些使用插值就足够了。
- (3) 使用光线步进计算内散射积分, 并在每条极线的内部做插值, 然后在不同的极线间做插值, 使内散射光计算覆盖到屏幕上的每个像素。

4.3.1 极线和初始样本生成

为了高效的做内散射积分和插值的计算, 我们把沿着极线方向的内散射光辐射度存储到一张二维纹理中: 每行表示一条极线, 每列表示极线上的一个样本点。如图 4-2 所示, 绿色的纹素表示极线的起始点和结束点, 黄色的纹素表示初始样本点, 而蓝色的纹素表示插值样本点, 纹理中列的数目决定了每条极线上光线步进样本的最大数量。我们并不需要对每个样本点都做光线步进计算, 因为插值样本点的内散射光辐射度可以通过初始样本点进行线性插值得到。

我们首先将二维空间的像素点与视口坐标关联起来, 并将信息存储到坐标系纹理中。为此我们将极线设置到 3.3 节中描述的像素着色器中: 每个纹理行都对应一条从光源投影点到某个屏幕边缘点的极线。

然后, 我们对这些极线按光锥体和视口裁剪, 留下确实对内散射有作用的部分, 剔除完全不可见的部分。

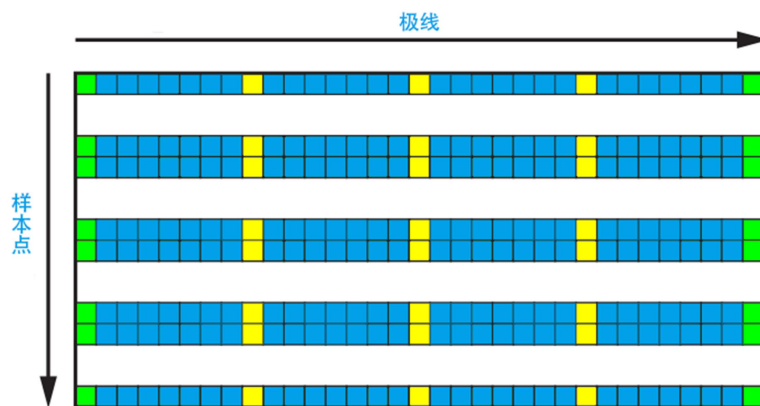


图 4-2 极线样本纹理

经过裁剪后，我们就可以得到每条极线的起点和终点，并以此来插值计算每个样本点的位置。在 3.3 节中我们曾提到，在视口外的极线片段将被从后续的计算中剔除，我们的方法是使用模板缓存遮罩，并通过 GPU 自动的将这些极线在接下来的渲染流程中剔除。

4.3.2 间断点检测和样本细化

在构建了极线之后，我们就可以决定哪些样本必须通过光线步进来做内散射积分计算（称作光线步进样本），哪些样本只需要靠插值计算就足够了（称作插值样本）。对于插值样本，我们还需要定义它应该根据哪两个样本点做内散射光插值。我们把该信息存储在另外一张二维纹理中（称作插值源纹理），该纹理的结构与 4.3.1 所描述的纹理相同。

在本文的实现中，我们将像素点的分类和插值源的定义放在了同一个像素着色器中。对插值纹理中的每个纹素，我们遍历它所属的整条极线片段（极线片段已经通过初始样本放置决定了）。对于极线片段中的每一个样本点，我们从场景的深度缓存中复制相应位置的深度值到一个本地数组中。

下一步，我们定义了极线片段中哪些样本将用来进行内散射光的插值计算，判断该像素是否会成为一个光线步进样本也会在该步骤决定。表 4-1 中的伪代码阐述了查找插值样本的过程（使用 left 和 right 标识），本地的深度数组为 `depth1d`，大小为 `N`，当前样本点在 `x`。（如图 4-3 所示）

表 4-1 插值样本点查询过程

```

left = right = x;
while ( left > 0 )
{
    if( abs( depth1d[left-1], depth1d[left] ) > threshold )
        break;
    left --;
}
while ( right < N-1 )
{
    if(abs( depth1d[right], depth1d[right+1] ) > threshold )
        break;
    right ++;
}
    
```

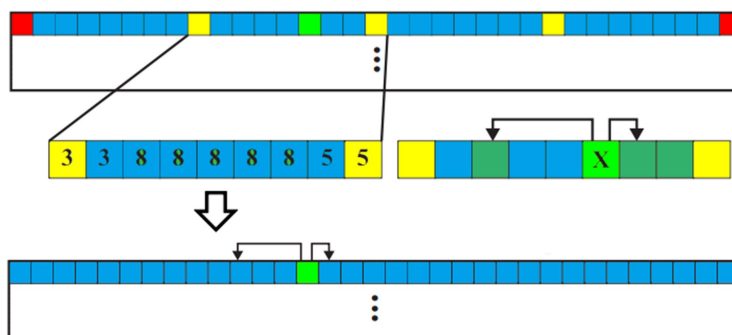


图 4-3 样本细化过程

4.4 构建一维最小/最大二叉树

我们将最小/最大二叉树存储为一个大小为 $SM_{Dim} \times (N_{Slices} \cdot N_{Cascades})$ ，格式为 RG_16UNORM 的纹理 tex2DMinMaxDepth，该纹理中包含了所有极切片的二叉树信息。就像在 3.4.1 中讨论的，每一个一维高度图都不会包含超过 SM_{Dim} 个样本，所以我们并不需要存储高度图本身，而是可以将所有的粗略层都封装进 $SM_{Dim} - 1$ 个样本中（假设阴影贴图维度是 2 的平方）。由于我们对数据进行了如上的封装处理，所以第一层的起始位置在列 $x=0$ 处，第 2 层的起始位置在列 $x=SM_{Dim}/2$ 处，以此类推。

二叉树的最深层（即最精细层）的数据通过从 `tex2DSliceUVDirAndOrigin` 纹理中读取切片起点 O_{UV} 和朝向 D_{UV} 来获得，并计算阴影贴图上的树结点所对应的样本点位置（ $O_{UV} + 2i \cdot D_{UV}$ 和 $O_{UV} + (2i+1) \cdot D_{UV}$ ）以及最小/最大深度信息。为了获取保守边缘，我们使用 `Gather()` 命令来获取四个深度，当进 PCF 滤波时，这将会对每一个样本位置使用^[32]。我们使用 16 位非标准化的纹理来存储数据，由于从 32 位浮点数转换到 16 位无符号整型可能丢失精度，在做精度转换时必须加以注意。

二叉树的上层（更粗略的层次）数据通过读取两个精细层节点的最小/最大边界来计算，并取这些数据的最小/最大值。为了达到这样的目的，我们使用两个纹理，将它们交替的设为源纹理和目标纹理。由于非常粗略的树层不太可能会达到，所以在实现中我们限制了最粗略的层次为阴影贴图分辨率的 1/16。这是因为将信息渲染到低分辨率的纹理在现代 GPU 中效率是比较低的，而且在进行过长的步骤后数值积分计算也会变得不精确。

4.5 光线步进

在构建了一维最小/最大二叉树后，下一步是计算极线上的每个样本点的内散射辐射度。我们将得到的结果存储在第三个纹理中（大小和 4.4 节所述的纹理相同），称作辐射度纹理。通过 4.3.2 小节所述的间断点检测步骤后，一些样本点被归类为插值点，而另一些被归类为光线步进点。然后，我们就需要开始执行代价昂贵的光线步进操作了，但我们并不希望 GPU 在不必要的样本上浪费时间，为此，我们可以使用模板缓存先将相应的光线步进样本标记出来。该步骤是在我们将整个屏幕的像素信息渲染到一张辐射度纹理后完成的，然后我们再使用一个像素着色器做光线步进计算，这样，GPU 的提前深度测试优化就可以使得这个着色器只在需要的时候执行（延迟着色）。

我们使用一个像素着色器来实现 3.4.2 节中所描述的光线步进算法。其中 `tex2DInitialInsctr` 纹理被设为渲染目标，而 `tex2DEpipolarStencil` 仅用作标识光线步进样本。我们仅列出其中最关键的着色器代码，如表 4-2 所示，注意注释部分。

表 4-2 光线步进着色器

```
// 计算射线终点、射线总长、和视线方向，并剔除在大气层之外的射线
for ( uint Cascade = g_StartCscd ; Cascade < g_NumCascades; ++Cascade )
{
```

```

// 剔除在最小/最大深度范围外的视线，并向阴影贴图中投影
// 加载切片的起点和朝向，分别在阴影贴图空间和世界空间计算积分
// 步骤 f3UVAndDepthStep，并设置正确的位置 f3CurrUVAndDepth
// 在一维高度图中计算初始样本位置
intiDataOffset = -g_iSMDim; // 我们并不存储第 0 层
while ( fMarchedDist < fRayLength )
{
    // 如果样本点处于适当的位置，那么就进一步计算下一个更粗略层
    if ( ( uiSamplePos & ( ( 2 << uiLevel ) - 1 ) ) == 0 ) {...}
    while ( uiLevel > 0 )
    {
        // 计算当前射线片段结尾的深度值
        // 为结点加载最小/最大深度值
        if ( IsInLight || IsInShadow )
            break ; // 如果射线片段是完全被照亮的或者完全处于阴影
                // 中的，我们就可以跳出循环
                // 如果射线不完全被照亮，也不完全处于阴影中，就必须到更精细层
    }
    if ( uiLevel == 0 )
    {
        // 如果已经在最精细的层次，那么就在 f3CurrUVAndDepth. xy 位置
        // 对阴影贴图使用 PCF 滤波
        IsInLight = ...
    }
    ...
    // 使用 fStep 作为步进因子，更新 f3RlghIn, f3MieIn, 和 f2NetDensFromCam
}
} // 应用瑞利散射和米氏散射的相位函数

```

表 4-2 所述像素着色器的输出结果，即是每个光线步进样本点的内散射积分计算结果，接下来我们就可以通过样本点的积分值对整个屏幕的像素做插值计算。

4.6 像素插值

最后，我们通过一个像素着色器来做插值：对于每个像素点，查找两个插值源，并根据相应的位置进行插值。注意我们可以使用之前已经构建了的模板缓存来区分样本类型，而在间断点检测中所有的光线步进样本都会将自其自身作为插值源。

至此，我们已经生成了决定最后图像所有像素点的内散射辐射度的所有必要信息，则可以进行最后的插值计算。首先我们计算从光源投影位置到像素点的射线，然后用屏幕边缘截断这条线。下一步我们决定靠近交点的最近极线。为了方便计算，我们使用一个一维纹理存储屏幕边缘上每个像素点的最近极线。注意该纹理的大小只取决于极线的数量，并且只会通过预计算存储一次。

最后，我们将像素投影到离它位置最近的两条极线上，从而获得插值信号。其实现如 3.4.4 小节所述：在沿着每条极线做线性插值后，我们在不同的极线间做插值。如果没有合适的信号，即没有在最近的极线上找到足够权重的信号，我们就进一步在下一条更靠外的极线上寻找合适的插值点。在本文的实现中，这样的处理已经足够找到一个合适的辐射度样本。

4.7 本章小结

在本章中，我们描述了本文的算法在 GPU 上的实现细节，并对其中一些关键之处作了详细的说明。首先我们介绍了我们的程序所使用的图形库 DirectX11 的特点，以及它的优势。然后我们从总体上描述了我们的实现过程，再针对极线采样与一维最小/最大二叉树中的一些实现细节进行了重点的阐述。

第五章 实验结果与讨论

在本章中，我们将通过一个测试程序实际展示我们的算法的渲染效果，以及渲染效率。

5.1 配置表选项

在测试程序中，我们提供了许多可配置的选项。通过这些选项，我们可以很方便的的对算法中的各个参数进行调节，从而做不同的对比与讨论。这些参数会对渲染画面和渲染效率产生不同的影响，我们要做的是在其中找到一个效果与效率间的平衡点，以发挥出优化算法的最大作用。

表 5-1 配置表选项说明

Full screen	—— 打开或关闭全屏模式。
Vsync	—— 打开或关闭垂直同步（锁定帧率为 60 帧）。
Enable light scattering	—— 打开或关闭大气散射效果。
Enable light shafts	—— 打开或关闭体积光渲染。若该选项被关闭，则我们不会进行可见因子的计算，即忽略阴影直接计算散射积分。
Scattering technique	—— 此配置有以下两个选项。
Epipolar sampling	—— 使用极线采样来计算散射积分。
Brute force ray marching	—— 对屏幕上每个像素点进行强制的散射积分计算。该模式可以产生“真实”的散射效果，主要用于和我们的极线采样算法进行对比。
Shadow map resolution	—— 该配置项提供阴影贴图分辨率大小的选择，范围为 512*512 到 4096*4096，增大此选项可以一定程度增强阴影的细致程度和场景的真实性，但也会较明显的影响渲染帧率。默认为 1024*1024。
Num integration step	—— 该配置项设定了计算单一散射时的步进次数，仅在 Enable light shafts（体积光渲染）项关闭时有效。默认为 30。
Epipolar slices	—— 该配置项设定了极切片的数目，范围为 32 到 2048。极切片的数量越多，渲染效果越真实，但相应的计算开销也会增大。如果希望得到比较好的显示效果，那么极切片数量应至少为显示分辨率的一半（例如 1280*720 的分辨率，则极切片数应在 512 到 1024 间）。默认为 1024。

Total samples in slice —— 该配置项设定了每条极线上的总样本数，范围为 32 到 2048。同样的，样本数量越多，渲染的效果越好，但计算开销会增大。较合适的数量为显示分辨率的一半（例如 1280*720 的分辨率，则每条极线的样本数应在 512 到 1024 间）。默认为 512。

Initial sample step —— 该配置项设定了初始光线步进样本的步骤数。步骤数越少，则每条极线上将会放置越多的初始样本，渲染效果就越真实，但同样会增加开销。默认为 16。

Epipole sampling density —— 该配置项决定了初始光线步进样本在近极核点处的增量，该选项在米氏散射强烈、光强变化明显时会产生较大的影响。默认为 2。

Refinement threshold —— 该配置项决定了细化精度。阈值越小，则会有更多的光线步进样本放置在深度不连续处，可以提供更好的渲染效果，但对渲染效率也会有较大的影响。

Scattering scale —— 该配置项决定了散射光的强度（注意在打开 HDR 中的自动曝光后，该项的作用效果不明显）。

Show sampling —— 该选项控制了极线的可见性。大的红点表示光线步进样本，小的红点表示插值样本。

1D Min/Max optimization —— 该选项控制了是否使用一维最小/最大二叉树来加速光线步进过程。

Optimize sample locations —— 该选项控制是否修复较短的极线处的过采样问题（3.3.2 节所述）。

Correction at depth breaks —— 该选项控制了是否在插值计算中无法精确计算的地方用额外的过程来校正样本。在该过程中，光线步进算法将对每个像素点都执行，一维最大/最小二叉树也无法对这些像素点使用，因为这些像素点并不属于任何极线。所以，光线步进计算在该过程中效率很低。但在一般的地形场景中，通常只有极少的像素需要进行校正，所以该选项可以安全的关闭而不会带来可见的视觉损失。

Show depth break —— 该选项可以将上述的校正过程使用绿色的像素点描述出来。该选项只在 **Correction at depth breaks** 选项设为真时有效。

Lighting only —— 该选项设定是否只显示光线散射效果（即关闭地形纹理）。

Single scattering mode —— 该配置项提供了以下三种选项。

Singl sctr: none —— 关闭单一散射计算。

Singl sctr: integration —— 使用直接数值积分进行内散射积分计算。

Singl sctr: LUT —— 使用预计算查找表进行内散射积分计算。如果设定了该选项，那么在光线步进过程中，我们就只需要计算被照亮的射线部分和，以及到第一个被照亮片段的距离，然后我们使用这两个参数作查找项对查找表进行查询，即可得到射线路径上的内散射光强。

Set Rayleigh color —— 该配置项可以设定瑞利散射光的颜色。

Set Mie color —— 该配置项可以设定米氏散射光的颜色。

Aerosol density —— 该配置项可以设定气溶胶的浓度，影响米氏散射的效果。

Aerosol absorption —— 该配置项可以设定气溶胶的吸收率，影响米氏散射的效果。

Animate Sun —— 该选项可以开启或关闭动态太阳运动。

5.2 渲染效果展示

接下来我们将通过调整上节所述的配置表选项，来展示算法的不同渲染效果。

首先我们将大气散射效果关闭，如图 5-1 所示。该场景中由于没有大气散射，从而天空是完全黑暗的，显得十分不真实。这很像是月球上的场景，因为月球上并没有大气层的存在。

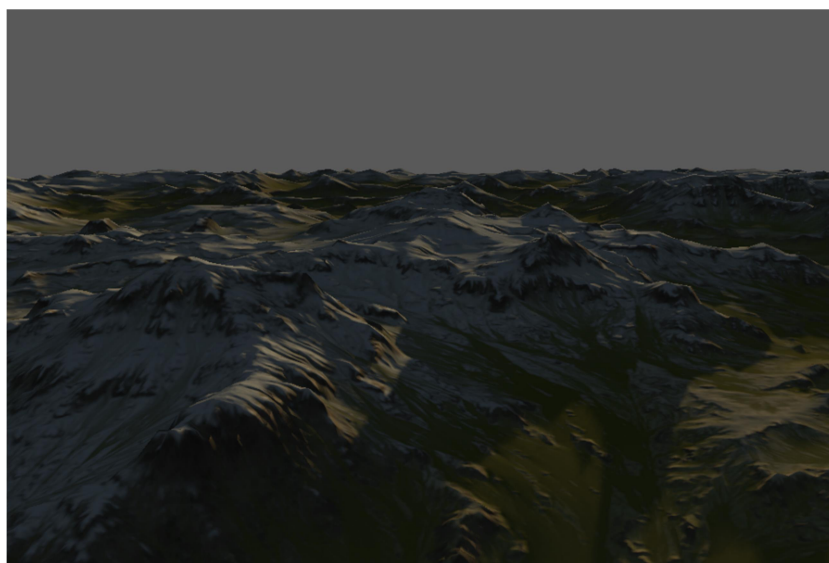


图 5-1 仅有直接光照的场景

然后我们打开大气散射效果，如图 5-2 所示。我们可以看到，整个场景的真实性大

幅度提高，天空由于瑞利散射的原因呈现渐变的蓝色，而在近地表处，由于米氏散射的原因，我们可以看到较明显的类似云雾的效果。

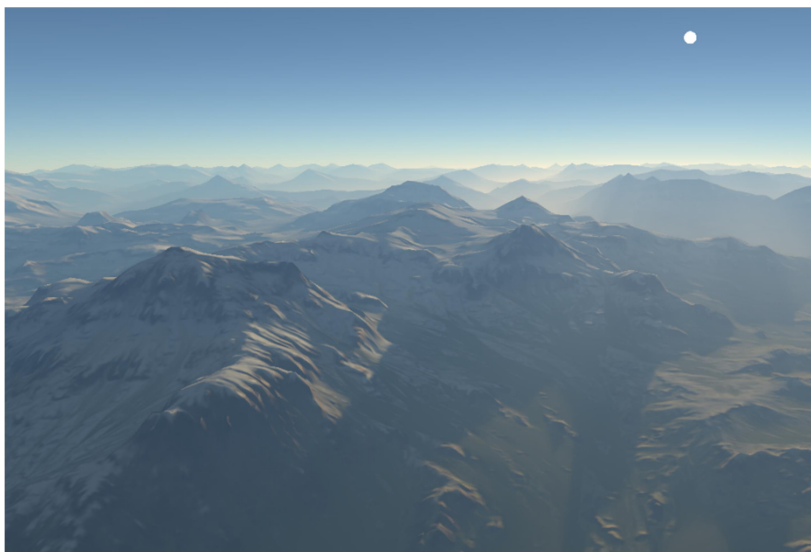


图 5-2 加入了大气散射后的场景

接下来我们调节太阳的高度，如图 5-3。可以看到，由于太阳光斜射入大气，在传播路径上的蓝色光大部分被散射而无法到达摄像机，所以越靠近地表，天空的颜色越接近橙黄色。并且，当阳光在穿过地表间的沟壑时，我们可以观察到明显的体积光现象。

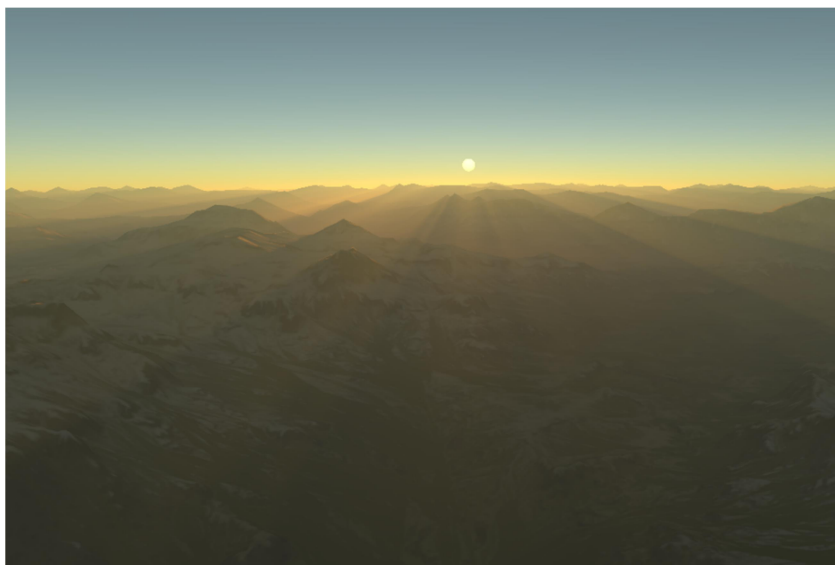


图 5-3 太阳在近地平面时的散射效果

我们可以将体积光关闭（即不考虑可见因子的影响），如图 5-4。可以看到，与图 5-3 的效果相比，没有了体积光的效果，场景的真实性下降了很多。

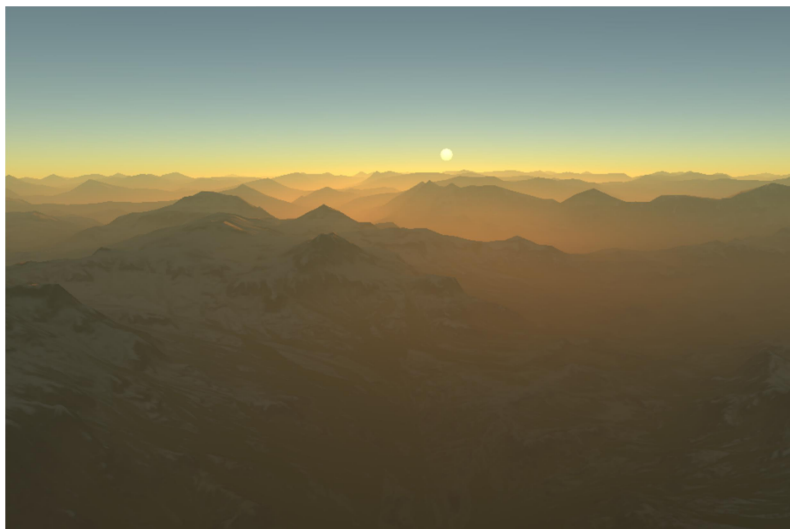


图 5-4 关闭体积光渲染的场景

我们分别使用极线采样（图 5-5 左）和逐像素点的光线步进（图 5-5 右）积分算法来渲染散射光，可以观察到在画面上二者只有细微的差别，但极线采样在渲染效率上会高出许多（在 5.3 节中将作展示）。

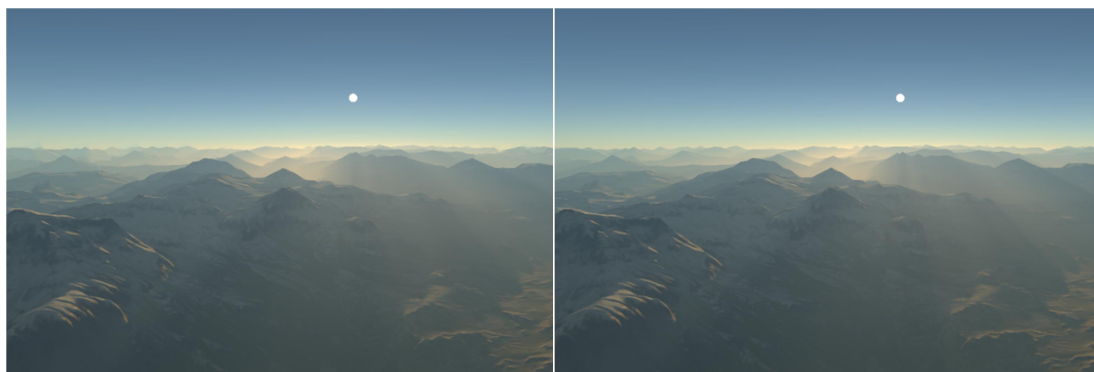


图 5-5 极线采样和逐像素点渲染画面对比

下面我们调节极切片的数量，观察不同切片数对渲染效果的影响。图 5-6 左中右分别是 32-128-1024 个极切片数时的场景，可以看到，过少的极切片会导致过稀疏的采样，从而使最终的渲染画面出现明显的人工痕迹。较合适的数值应至少为屏幕分辨率的一半，但注意过多的切片数会带来更大的计算开销。

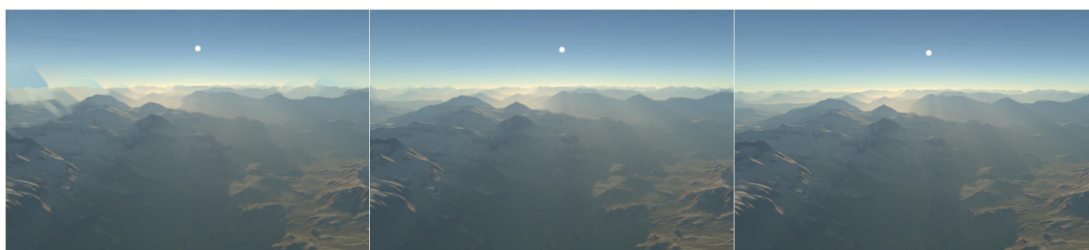


图 5-6 不同极切片数的采样渲染效果

接下来我们调节每条极线上的样本数量，观察不同样本数对渲染效果的影响。图 5-7 左中右分别是每条极线进行 32-128-1024 次采样的场景，可以看到，过少的采样数会导致明显的条状失真。比较合适的样本数应为屏幕分辨率的一半，但过多的采样数同样会带来额外的计算开销。

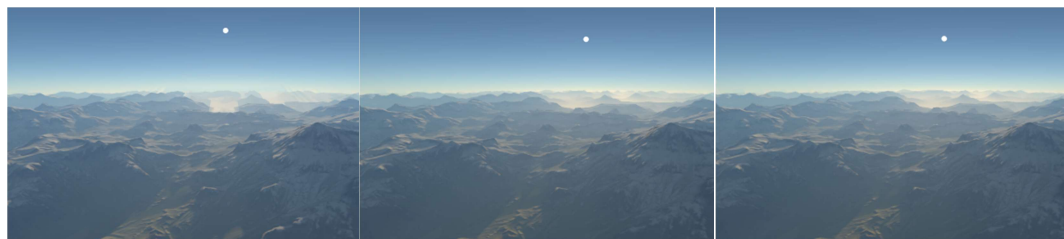


图 5-7 不同样本数的渲染效果

然后我们调节不同数量的初始光线步进样本数量，观察对渲染效果的影响。图 5-8 上为 Initial sample step 设为 64 时的天空效果，可以看到天空中有明显的不自然的光晕，这是采样过疏导致的结果。而图 5-8 下为 Initial sample step 设为 1 时的天空效果，可以看到天空的颜色变得更为自然而真实，但在效率上会导致明显的下降。

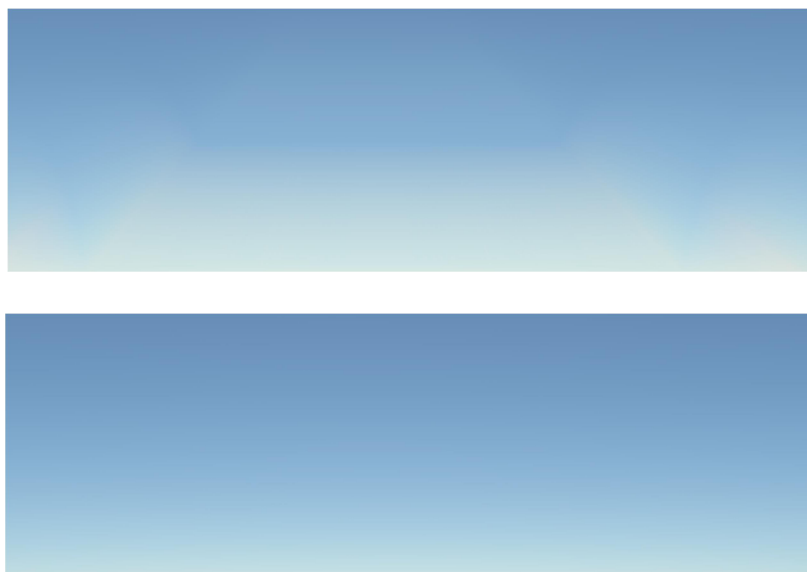


图 5-8 不同初始光线步进样本数量的渲染效果

我们接下来调整样本细化精度。可以看到，当细化精度较低时，在近地表时由于场景深度值变化较大，所以会出现明显的渲染错误（图 5-9 上），而在提高细化精度后，这种失真现象会逐渐消失（图 5-9 中和下），但也会带来较明显的性能损耗。

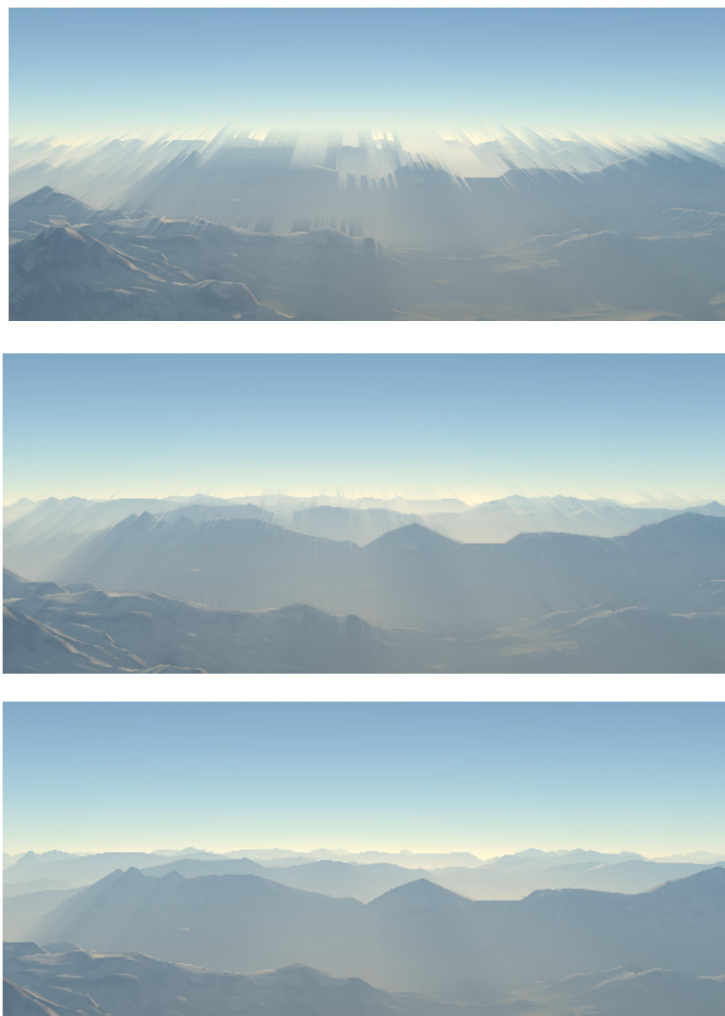


图 5-9 不同样本细化精度的渲染效果

为了测试散射光强度对渲染效果的影响，我们首先需要将高动态范围光照中的自动曝光功能关闭。可以看到，散射光强度的大小决定了整个场景的明亮程度（图 5-10）。该效果对渲染性能并无影响，但适当的散射光强可以产生更真实的画面。

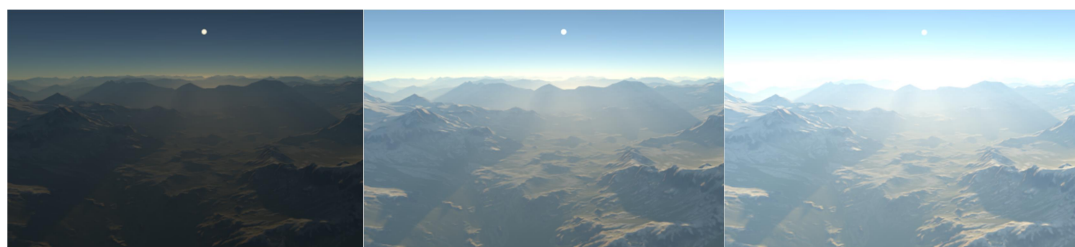


图 5-10 不同散射光强度的渲染效果

同时，为了更清楚的观察散射光效果，我们还可以将地形渲染关闭，从而只对散射光进行观察，如图 5-11。



图 5-11 关闭地形渲染后的散射光效果

我们还可以对气溶胶的浓度做调整，它影响了空气中大颗粒米氏散射的强度，从而可以模拟空气干净或浑浊时的不同渲染效果。如图 5-12 左，干净的空气会受更多瑞利散射的影响，从而远处的场景会显示出淡蓝色，而在图 5-12 右中，空气中大颗粒浓度较高，则显示出类似云雾的效果。

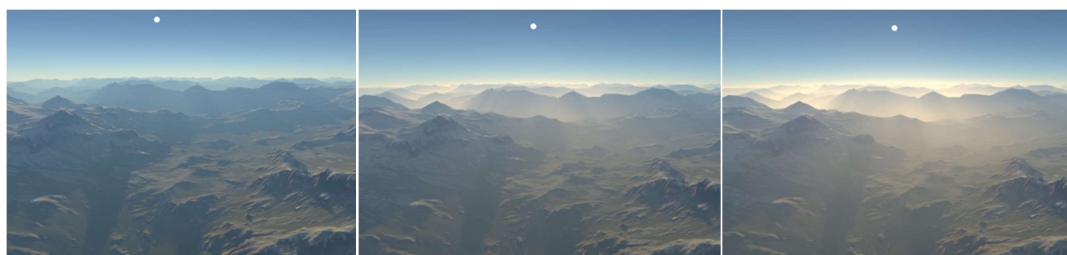


图 5-12 不同气溶胶浓度的渲染效果

另外，我们还可以对瑞利散射和米氏散射光的颜色进行手动设定，尽管这样会明显降低场景的真实性，但在一些特殊的应用场合下(如我们想要模拟出一个外星球的场景)，仍有相应的作用。如图 5-13 左，瑞利散射被设为红色，米氏散射被设为蓝色。中，瑞利散射被设为绿色，米氏散射被设为红色；右，瑞利散射被设为蓝色，米氏散射被设为红色。

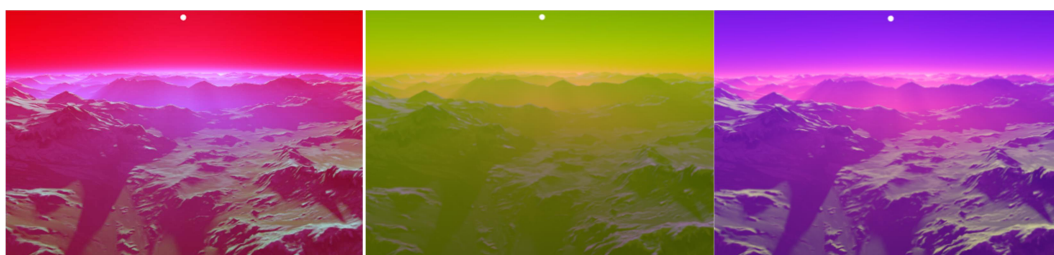


图 5-13 不同散射光颜色下的渲染效果

5.3 渲染效率测试

本节中我们将对算法的渲染性能进行测试，观察并讨论得到的结果。

我们使用两台机器分别用于测试高配置与低配置下的渲染性能，测试环境如表 5-2 所示。

表 5-2 测试机器软硬件环境

		机器 1（高配置）	机器 2（低配置）
CPU	型号	Intel Core i7-6700	Intel Core i5-3470
	核心数	4	2
	主频	3.40G Hz	3.20G Hz
内存	大小	16GB	8GB
	规格	DDR4	DDR3
显卡	芯片	NVIDIA GeForce GTX 1060	NVIDIA GeForce GT 630
	显存	6GB	1GB
操作系统		Microsoft Windows 7 Ultimate	Microsoft Windows 7 Ultimate

我们分别使用以下六种不同的算法：

A. 每像素强制光线步进散射积分计算。这是一种最原始的方法，没有进行任何的优化和加速的操作，渲染效率很低，只用作数据对比使用。

B. 对每像素使用一张预计算的查找表来查询散射积分的值，从而降低散射积分计算昂贵的性能开销。这与 O'Neil 在 2004 年所提出的方法类似^[18]。

C. 使用极线采样来降低样本点的数量，但对于每个样本点，仍使用强制的散射积分计算，这就是 Engelhardt 和 Dachsbacher 在 2010 年所提出的通过极线采样计算体积光的方法^[23]。相对于对屏幕上的每像素点都做光线步进计算，该算法极大的降低了计算量，从而使渲染帧率得到了成倍的提升。

D. 使用极线采样加查找表的方法，该算法是算法 B 和算法 C 的结合，在进行极线采样后，对每个样本点再进行查找表操作，从而在[23]算法的基础上，进一步提升了渲染效率。

E. 在极线采样的基础上，使用一维最小/最大二叉树来加速可见因子的计算过程，这是 Chen 等人在 2011 年所提出的方法^[24]。由于不需要再对阴影贴图纹理进行遍历操作，

该算法进一步提高了渲染效率。注意在关闭 **light shaft** 选项后，该优化将失去作用。

F. 结合 D 和 E 方法，使用极线采样来降低采样点的数量，使用查找表来简化散射积分的计算，使用一维最小/最大二叉树来加速光线步进的计算过程，这也正是本文所使用的方法。

我们首先在高配置机器 1 上做测试，分别在 1280*720 和 1920*1080 两种分辨率下测试六种算法的渲染性能，结果如表 5-3 所示（单位为帧/每秒）。

表 5-3 高配置机器下的渲染性能比较

	1280*720	1920*1080
每像素	95	45
每像素+查找表	214	105
极线采样	455	327
极线采样+查找表	663	453
极线采样+一维二叉树	625	458
极线采样+查找表+一维二叉树	727	520

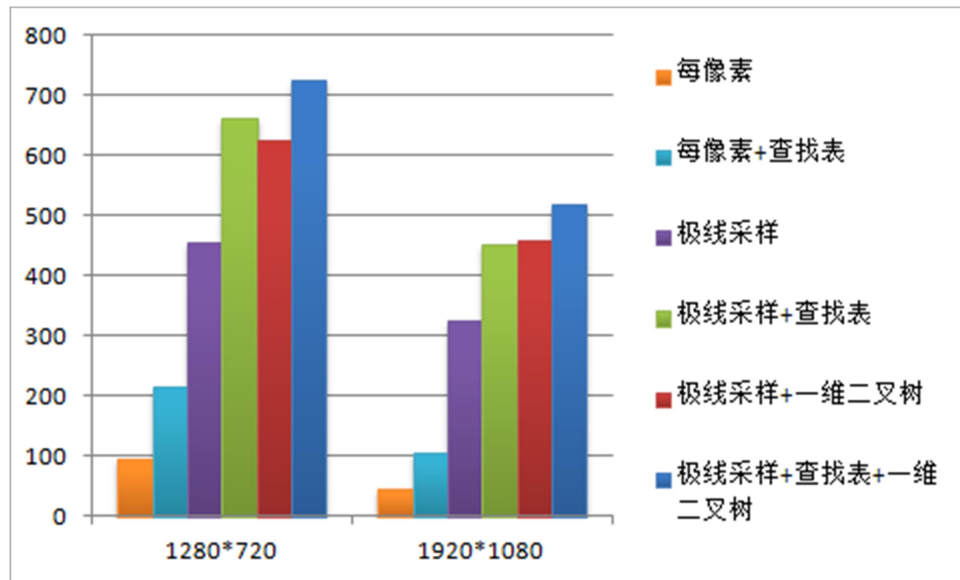


图 5-14 高配置机器下的渲染性能比较

经比较，在高配置机器中，本文的算法相对于每像素的强制光线步进散射积分计算方法^[10]，在 1280*720 的分辨率下，性能提高了 665%，在 1920*1080 的分辨率下，性能提高了 1056%。相对于 O' Neil 的查找表方法^[18]，在 1280*720 的分辨率下，性能提高了

339%，在 1920*1080 的分辨率下，性能提高了 495%。相对于 Engelhardt 和 Dachsbacher 的极线采样方法^[23]，在 1280*720 的分辨率下，性能提高了 60%，在 1920*1080 的分辨率下，性能提高了 59%。相对于 Chen 等人的方法^[24]，在 1280*720 的分辨率下，性能提高了 16%，在 1920*1080 的分辨率下，性能提高了 13%。

可以看到，现代的 GPU 计算能力已经十分的强大了，即使是使用最原始算法，依然可以得到较为流畅的结果。但实际的应用中（如大型的 3D 游戏），GPU 还需要将大量的计算量用于应用中其他内容的渲染，所以，算法所进行的优化，还是十分有必要的。

接下来我们在低配置机器 2 上做测试，观察并讨论测试的结果。我们同样分别在 1280*720 和 1920*1080 两种分辨率下测试六种算法的渲染性能，结果如表 5-4 所示（单位为帧/每秒）。

表 5-4 低配置机器下的渲染性能比较

	1280*720	1920*1080
每像素	8	4
每像素+查找表	15	7
极线采样	30	25
极线采样+查找表	46	34
极线采样+一维二叉树	43	32
极线采样+查找表+一维二叉树	55	38

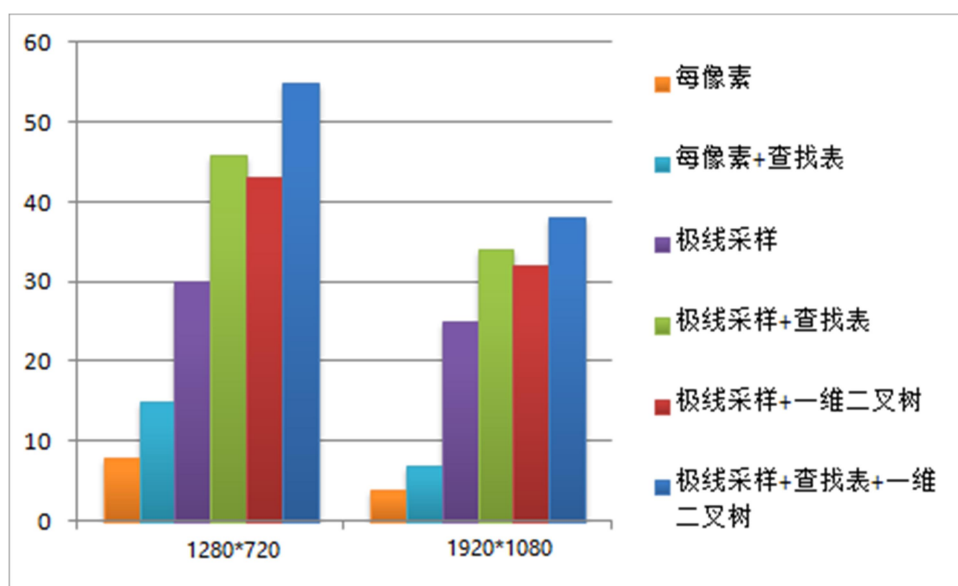


图 5-15 低配置机器下的渲染性能比较

经比较,在高配置机器中,本文的算法相对于每像素的强制光线步进散射积分计算方法^[10],在 1280*720 的分辨率下,性能提高了 587%,在 1920*1080 的分辨率下,性能提高了 850%。相对于 O' Neil 的查找表方法^[18],在 1280*720 的分辨率下,性能提高了 367%,在 1920*1080 的分辨率下,性能提高了 442%。相对于 Engelhardt 和 Dachsbacher 的极线采样方法^[23],在 1280*720 的分辨率下,性能提高了 83%,在 1920*1080 的分辨率下,性能提高了 52%。相对于 Chen 等人的方法^[24],在 1280*720 的分辨率下,性能提高了 28%,在 1920*1080 的分辨率下,性能提高了 19%。

可以看到在低配置机器上,不做任何优化直接进行散射积分的计算,是完全不可行的,在两种分辨率下帧数都已经低到了个位数,这在实时渲染中是不可接受的。经过我们的算法优化后,在 1280*720 的分辨率下,渲染帧率已经可以达到了 55 帧每秒,已经基本能够满足实时渲染的需求。

5.4 本章小结

在本章中,我们展示了算法的实际渲染效果和效率。首先我们介绍了测试程序中所提供的配置项,通过这些配置项可以很方便的调节渲染效果,同时影响到渲染效率。然后我们分别对配置项的参数做不同的设定,观察其对最终渲染画面的影响,选择其中最优的配置参数。最后,我们对本文算法的渲染效率做出了测试,通过与前人算法的直观比较,证明了本算法的优势。

总结与展望

大气散射是自然界中的一个普遍而又重要的物理现象，它直接影响了人们对室外场景的感知，例如天空在不同时间与不同气候下的颜色变化（晴朗的天空白天呈现蓝色，傍晚则呈现橙红色，而阴霾的天空呈现灰色），以及体积光的现象（云隙光、探照灯光束等）。所以，在计算机中模拟这种现象，对增强虚拟场景的真实性，具有十分重要的作用。

本文所采用的大气建模是完全基于物理的，综合考虑到了不同粒子密度对不同波长光线的影响，并采用从摄像机开始的光线步进方法计算太阳光的內散射积分，从最大程度上的模拟了真实的物理现象。同时，本文所实现的测试程序还可以进行动态的参数调整，可以分别模拟出不同时间段、不同空气质量下的大气散射效应。

对内散射积分的计算是一个十分复杂的过程，即使在现今的高性能 GPU 上依然很难进行直接的实时渲染。为此，本文使用了许多方法来进行渲染性能上的优化。首先我们使用一张预计算的二维查找表来简化散射积分的计算过程，经测试该方法可以平均将渲染速度提高 125%；然后我们在此基础上再进行极线采样来减少屏幕样本点的数量，经测试该方法可以平均将渲染速度进一步提高 209%；最后我们再使用一维最小/最大二叉树来加速光线步进中可见因子的计算，经测试该方法可以平均将渲染速度进一步提高 37%。最终，与最初始的每像素强制散射积分计算相比，总渲染效率平均提高了 665%。

同时，本文还有许多不足和值得改进的地方，例如本文的散射光模型遵循单一散射原理，即假设太阳光只会被空气粒子做一次散射，这在大部分时候可以提供比较真实的渲染结果和较快的渲染效率。然而，真实的物理光照却是遵从多重散射原理的，即被一个粒子散射出的光线还会进一步被其他的粒子散射，这就大大增加了问题的复杂性，并且使得其很难在实时渲染中实现。多重散射效果在白天的影响比较微小，但在黄昏时则可以比较明显的增加场景的真实性，所以如何将多重散射应用到我们的算法中，是我们将来的一个十分重要的研究课题。另外，尽管本文的算法可以模拟出雾与霾等天气效果，但却暂时还没有加入与云层的交互，如何将大气散射效果与云层渲染结合起来，这也是本文在将来的研究方向之一。

参考文献

- [1] Bouknight W J. A procedure for generation of three-dimensional half-toned computer graphics presentations[J]. Communications of the Acm, 1970, 13(9):527-536.
- [2] Gouraud, Henri. Computer display of curved surfaces[J]. 1971:35-41.
- [3] Phong, Tuong B. Illumination for computer-generated images.[J]. 1973.
- [4] Nishita T, Nakamae E. Continuous tone representation of three-dimensional objects illuminated by sky light[J]. ACM SIGGRAPH Computer Graphics, 1986, 20(4):125-132.
- [5] Nishita T, Miyawaki Y, Nakamae E. A shading model for atmospheric scattering considering luminous intensity distribution of light sources[C]// Conference on Computer Graphics and Interactive Techniques, SIGGRAPH. DBLP, 1987:303-310.
- [6] Yoshinori D, Kazufumi K, Takanobu N, et al. Skylight for Interior Lighting Design[C]// Computer Graphics Forum. Blackwell Science Ltd, 1994:85-96.
- [7] Klassen R V. Modeling the effect of the atmosphere on light[J]. Acm Transactions on Graphics, 1987, 6(3):215-237.
- [8] Inakage M. An Illumination Model for Atmospheric Environments[M]// New Advances in Computer Graphics. Springer Japan, 1989:533-548.
- [9] Kaneda K, Okamoto T, Nakamae E, et al. Photorealistic image synthesis for outdoor scenery under various atmospheric conditions[J]. The Visual Computer, 1991, 7(5):247-258.
- [10] Nishita T, Sirai T, Tadamura K, et al. Display of the earth taking into account atmospheric scattering[C]// Conference on Computer Graphics and Interactive Techniques. ACM, 1993:175-182.
- [11] Irwin J. Full-Spectral Rendering of the Earth's Atmosphere using a Physical Model of Rayleigh Scattering[J]. 1996.
- [12] Jackel D, Walter B. Modeling and Rendering of the Atmosphere Using Mie-Scattering[C]// Computer Graphics Forum. 1997:201-210.
- [13] Hoffman N, Studios W, Preetham A J. Rendering Outdoor Light Scattering in Real Time[C]// 2002.
- [14] Riley K, Ebert D S, Kraus M, et al. Efficient Rendering of Atmospheric Phenomena[M].

The Eurographics Association, 2004.

[15] Preetham A J, Shirley P, Smits B. A practical analytic model for daylight[C]// Conference on Computer Graphics and Interactive Techniques. ACM Press/Addison-Wesley Publishing Co. 1999:91-100.

[16] Mitchell K. Volumetric Light Scattering as a Post Process[J]. Gpu Gems, 2007.

[17] Dobashi Y, Yamamoto T, Nishita T. Interactive rendering of atmospheric scattering effects using graphics hardware[C]// ACM Siggraph/eurographics Conference on Graphics Hardware. Eurographics Association, 2002:99-107.

[18] O' Neil S. Real-Time Atmospheric Scattering[J]. 2004

[19] O' Neil S. Accurate Atmospheric Scattering[J]. Gpu Gems, 2005.

[20] Christian Schuler. “An Approximation to the Chapman Grazing Incidence Function for Atmospheric Scattering.” In GPU Pro 3, edited by Wolfgang Engel, pp. 105–118. Natick, MA: A K Peters, 2012.

[21] Schafhitzel T, Falk M, Ertl T. Real-Time Rendering of Planets with Atmospheres[J]. Václav Skala - UNION Agency, 2007, 15(2pt2):91-98.

[22] Bruneton E, Neyret F. Precomputed Atmospheric Scattering[J]. Computer Graphics Forum, 2010, 27(4):1079-1086.

[23] Engelhardt T, Dachsbacher C. Epipolar sampling for shadows and crepuscular rays in participating media with single scattering[C]// Symposium on Interactive 3d Graphics, Si3d 2010, February 19-21, 2010, Washington, Dc, Usa. DBLP, 2010:119-125.

[24] Chen J, Baran I, Durand F, et al. Real-time volumetric shadows using 1D min-max mipmaps[C]// Symposium on Interactive 3D Graphics and Games. ACM, 2011:PAGE@7.

[25] Egor Yusov. “High Performance Outdoor Light Scattering using Epipolar Sampling.” In GPU Pro 5: Advanced Rendering Techniques, edited by Wolfgang Engel, pp. 101–126. Boca Raton, FL: CRC Press, 2014.

[26] SOUSA T. Crysis next gen effects. Game Developers Conference 2008 Presentations, 2008.

[27] Hartley R, Zisserman A. Multiple view geometry in computer vision[J]. Kybernetes, 2003,

30(9/10):1865 - 1872.

[28] Williams L. Casting curved shadows on curved surfaces[J]. Acm Siggraph Computer Graphics, 1978, 12(3):270-274.

[29] Tevs A, Ihrke I, Seidel H P. Maximum mipmaps for fast, accurate, and scalable dynamic height field rendering[C]// Symposium on Interactive 3d Graphics and Games. ACM, 2008:183-190.

[30] Reeves W T, Salesin D H, Cook R L. Rendering antialiased shadows with depth maps[C]// Conference on Computer Graphics and Interactive Techniques, SIGGRAPH. 1987:283-291.

[31] Wyman C, Ramsey S. Interactive volumetric shadows in participating media with single-scattering[C]// Interactive Ray Tracing, 2008. RT 2008. IEEE Symposium on. IEEE Xplore, 2008:87-92.

[32] Egor Yusov. Practical Implementation of Light Scattering Effects Using Epipolar Sampling and 1D Min/Max Binary Trees[C]// Presentation, Game Developers Conference 2013, San Francisco, CA, March, 2013.

攻读硕士学位期间取得的研究成果

一、已发表（包括已接受待发表）的论文，以及已投稿、或已成文打算投稿、或拟成文投稿的论文情况（只填写与学位论文内容相关的部分）：

序号	作者（全体作者，按顺序排列）	题 目	发表或投稿刊物名称、级别	发表的卷期、年月、页码	相当于学位论文的哪一部分（章、节）	被索引收录情况

注：在“发表的卷期、年月、页码”栏：

- 1 如果论文已发表，请填写发表的卷期、年月、页码；
 - 2 如果论文已被接受，填写将要发表的卷期、年月；
 - 3 以上都不是，请据实填写“已投稿”，“拟投稿”。
- 不够请另加页。

二、与学位内容相关的其它成果（包括专利、著作、获奖项目等）

致谢

研究生三年的生活如白驹过隙，转瞬即逝，如今已经接近了尾声。回顾这三年，有成功也有挫折，有喜悦也有失落，但更重要的是，这些都让我在这三年中得到了不断的成长，不仅在学术水平上有了很大的提高，在为人处事方面同样也成熟了许多。为此，我希望在这里对帮助过我的人做出由衷的感谢。

首先，我最希望感谢的是我的导师，陈泽琳老师，她是一个十分高尚的人，无论在学术造诣上，还是个人修养上，都非常令我敬佩。陈老师治学严谨细心，为人随和热情，在这三年来，给了我许多的帮助，不仅是在学习和科研上，更是在生活的各个方面上。陈老师，能够成为您的学生，我感到十分的荣幸，您的每个品质都值得我用一生去学习。

然后，我要感谢我的同学们，正是有他们与我一起共同学习、共同进步，我才能得到今天的成长。很高兴能遇见你们，并能和你们在一起度过这三年的时光。

当然，我还要感谢我的父母和家人，正是因为他们一直在背后对我的支持和鼓励，才让我在遇到挫折时没有放弃，给了我继续前进的动力。感谢你们，你们永远是我学习的榜样，也是我最坚实的后盾。

最后，感谢参加对本文审阅的各位专家、老师，感谢你们在百忙之中抽出时间，谢谢！

答辩委员签名的答辩决议书

答辩委员会对论文的评定意见

方辰同学的硕士学位论文“基于 GPU 的实时大气散射渲染优化算法研究与实现”有较好的实际意义和应用价值。

论文参照了较多的相关文献资料,实现了一个基于 GPU 计算的实时大气散射渲染优化算法。通过对大气进行基于真实物理的建模,利用光线步进的思想计算散射积分,并使用预计算查找表来简化散射积分计算、通过极线采样来降低样本点的数量、利用一维最小/最大二叉树结构来加速可见因子的判断。算法在保证真实的画面渲染效果的同时,显著的提升了渲染效率。

作者对文献的调研、数据的收集较充分,论文资料丰富,论点正确,结论合理,实验数据真实可信。反映作者具有较强的理论基础、较好的科研和工程能力。

论文结构清晰,图文表达规范,达到硕士学位论文水平。答辩过程讲述清楚,回答问题正确。经答辩委员会无记名投票,同意该同学通过硕士学位论文答辩,同意授予硕士学位。

论文答辩日期: 2017 年 6 月 2 日

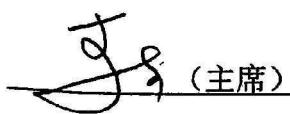
答辩委员会委员共 5 人,到会委员 5 人

表决票数:优秀(0)票;良好(5)票;及格(0)票;不及格(0)票

表决结果(打“√”) :优秀() ;良好(√) ;及格() ;不及格()

决议:同意授予硕士学位(√) 不同意授予硕士学位()

答辩
委员
会成
员签
名

 (主席)

